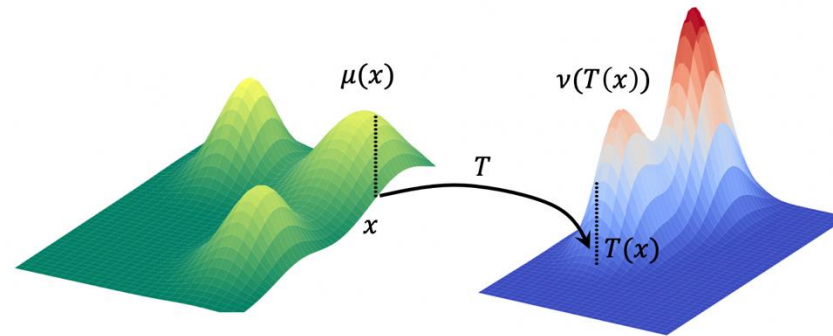


Optimal Transport for Machine Learning

ADVANCED MACHINE LEARNING

Machine Learning and Data Mining Master Degree

Marc SEBBAN



Note: This tutorial takes inspiration from several tutorials on optimal transport given by (non exhaustive list):
M. Cuturi, J. Solomon, G. Peyré, N. Courty, R. Flamary, S. Kolouri, K. Rohde, etc.

Probability distributions in **Image Processing**

RGB images as 3D distributions of colors

How to measure the
(dis)similarity
between 2 Images?

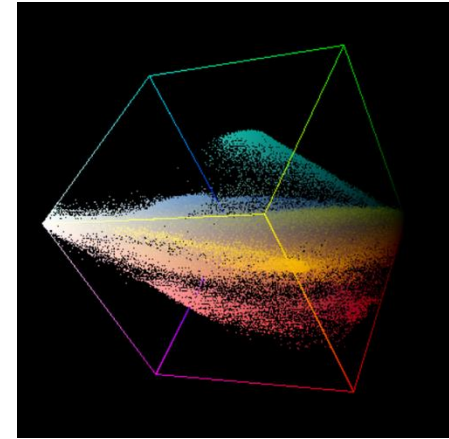


Credits: Patrick David

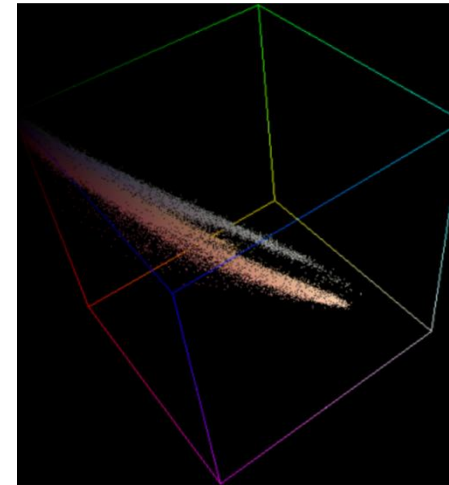


Credits: Patrick David

Clouds of 3D points



Credits: David Tschumperlé

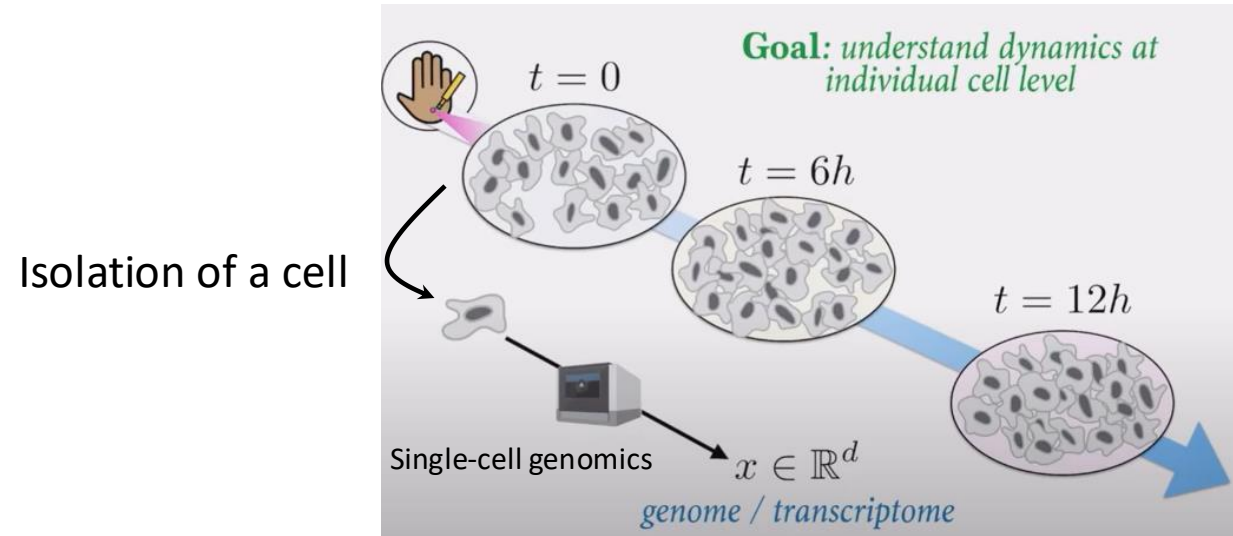


Credits: David Tschumperlé

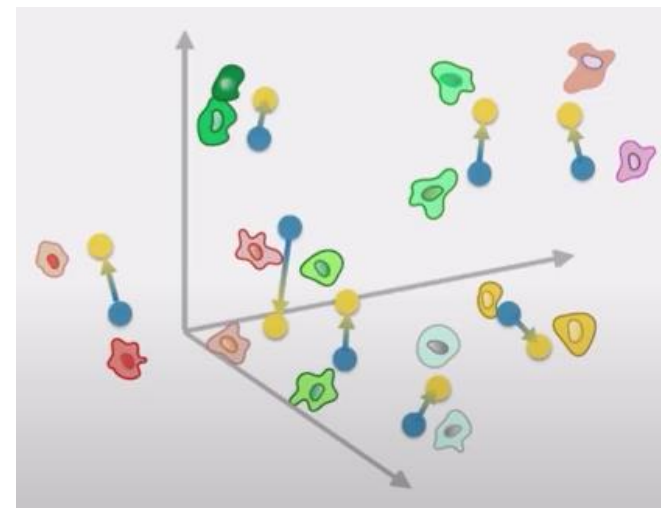
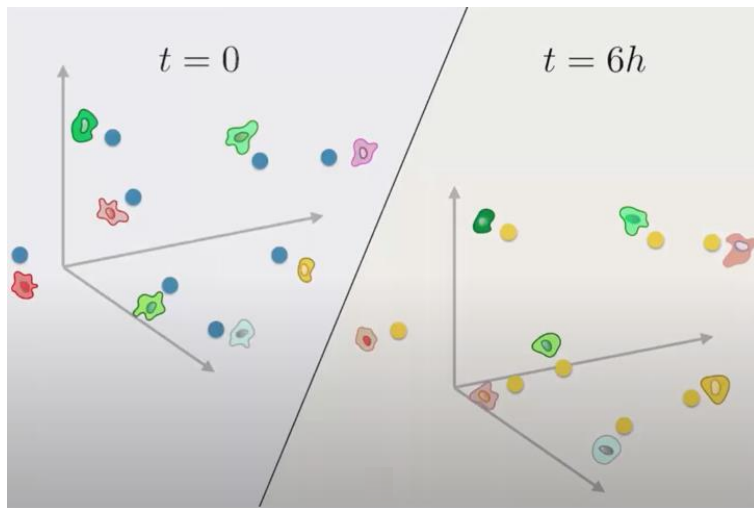
By computing, e.g., a
distance/divergence
between the two
3D-distributions

... in Genomics

Human tissue as distributions of cells



How to estimate the dynamics of the cells?

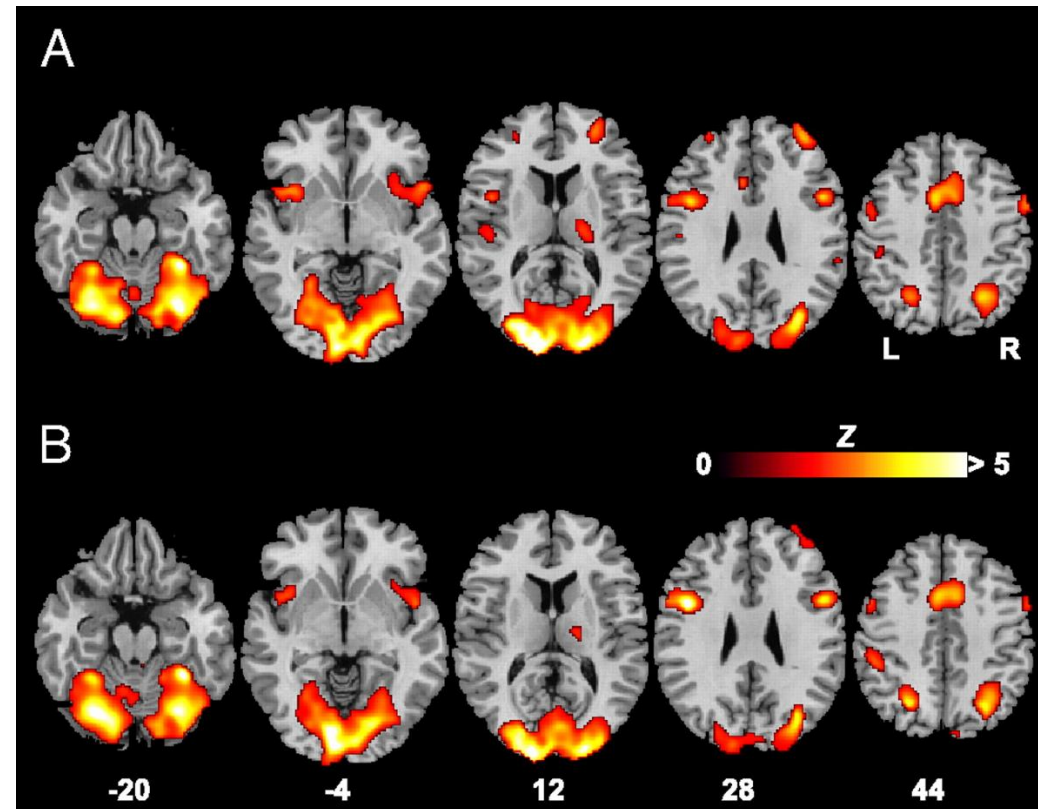


By aligning the 2 empirical distributions in the cell embedding space

... in Brain Imaging

Brain activation maps as probability measures

How to compare 2 brain activation maps?

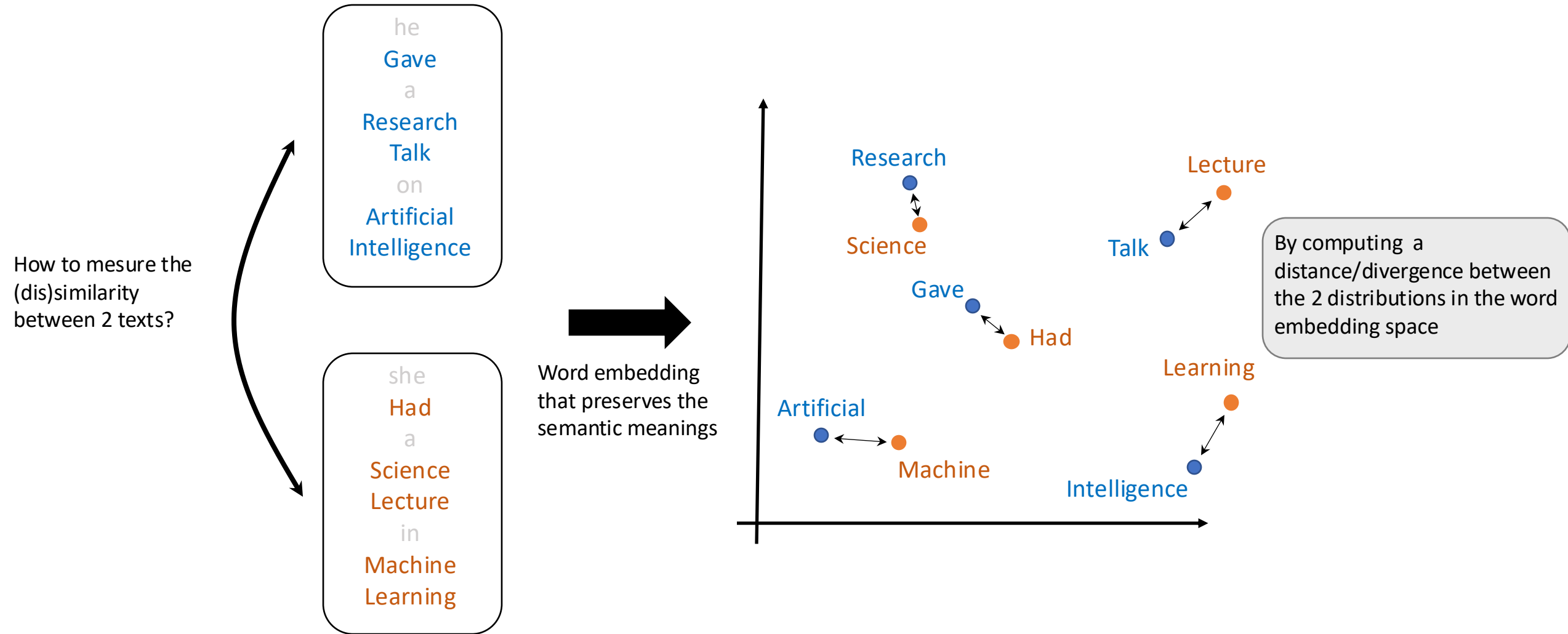


By computing a distance/divergence between the 2 distributions

Li Hai Tan et al. 2008

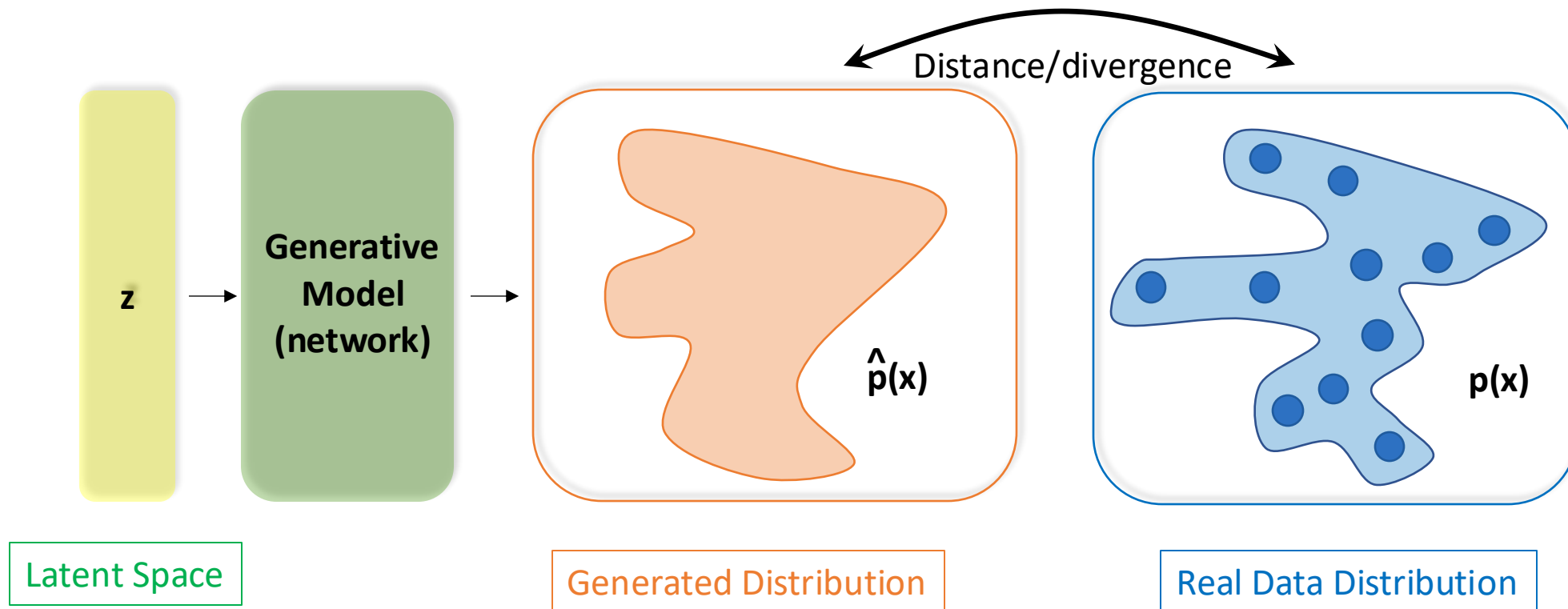
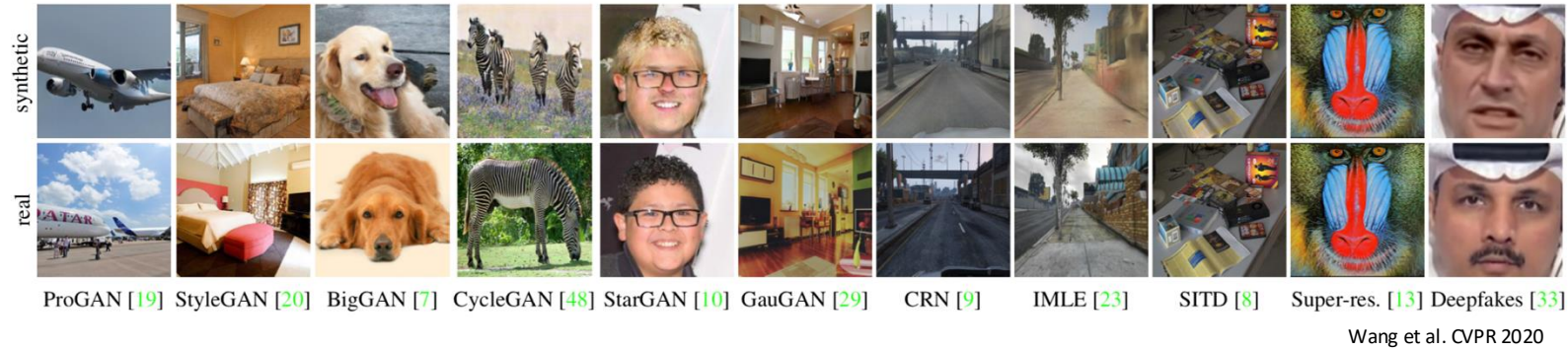
... in Natural Language Processing

Texts as distributions of words

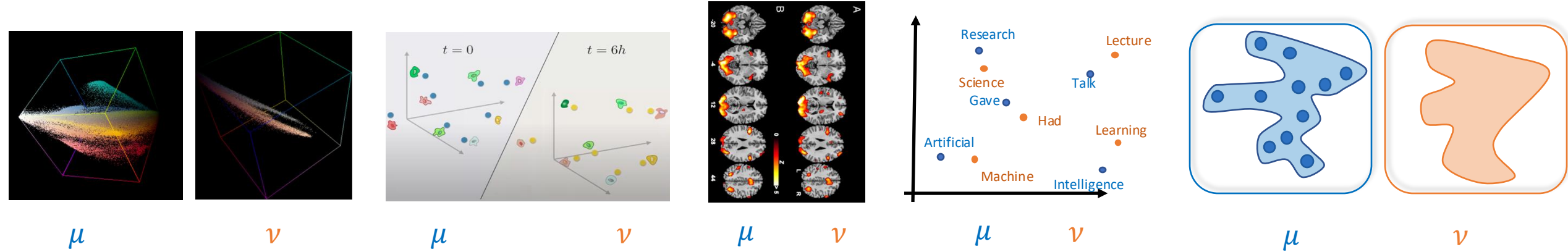


... in Deep Learning

Generative Models versus Data



Need to define a geometry in the space of probability measures



How to compute a distance between μ and ν ?

How to align μ and ν ?

How to compute an interpolation between μ and ν which preserves the geometry?

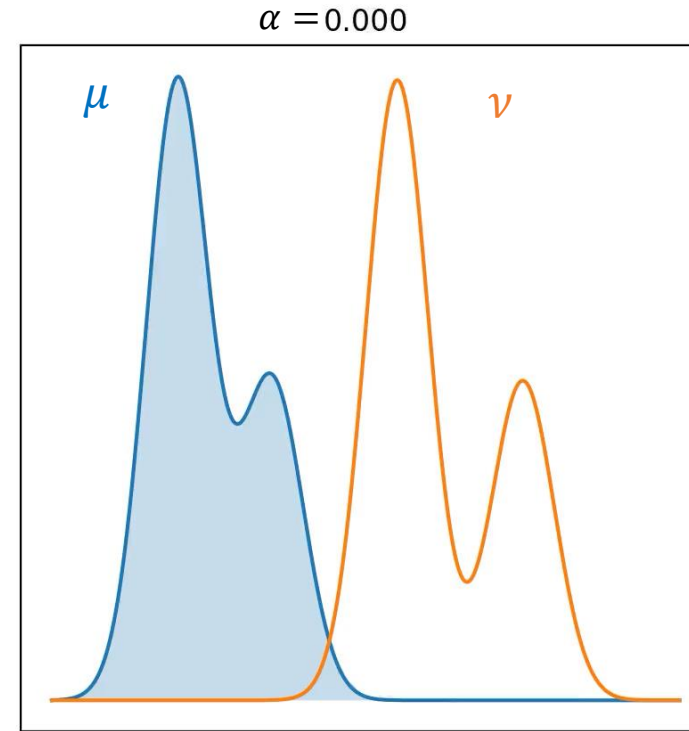
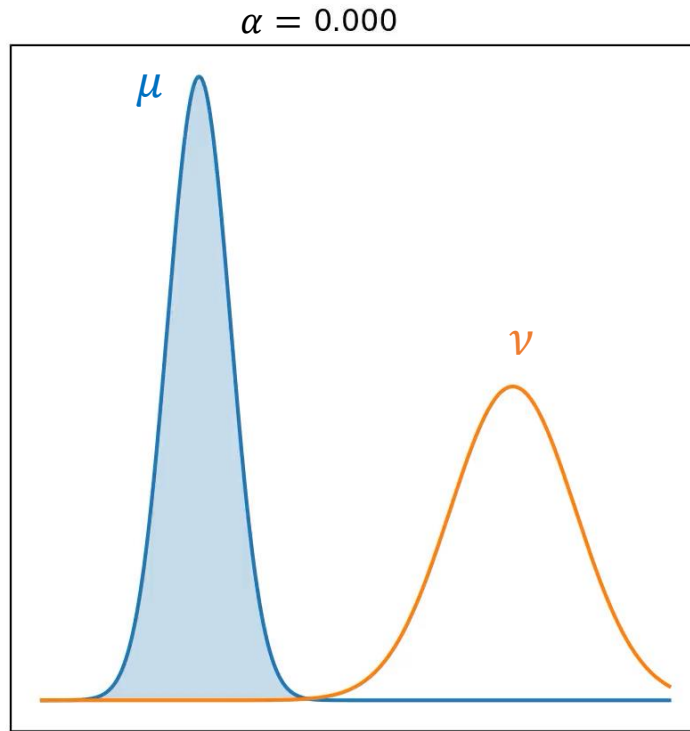
What are the main limitations of the classic distances/divergences like L1, L2, Kullback-Leibler?

Limitations of standard distances/divergences

$$(1 - \alpha)\mu + \alpha\nu ?$$

L_2 - Interpolation

KL - Interpolation



Credits: Rémi Emonet

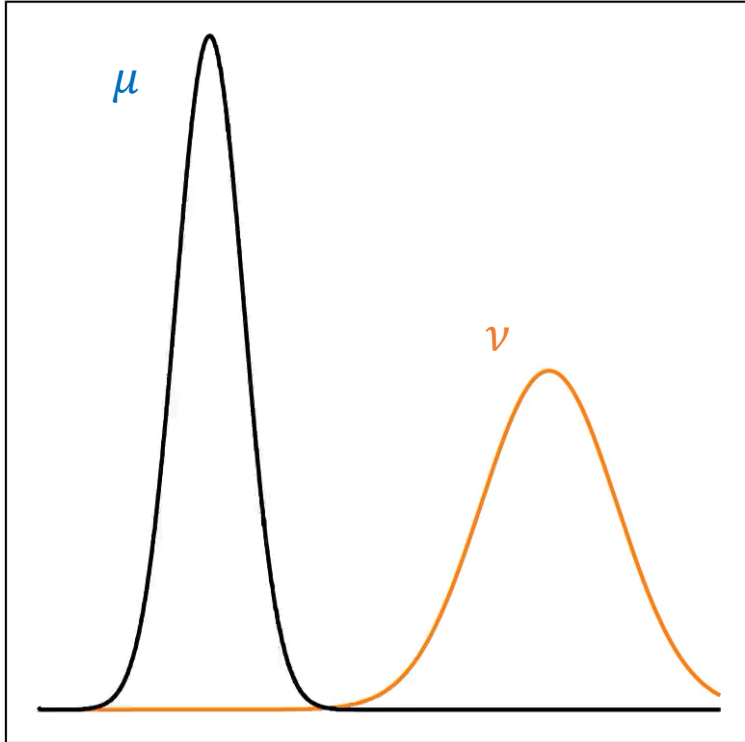


From a geometry perspective, the interpolation does not preserve the underlying structure

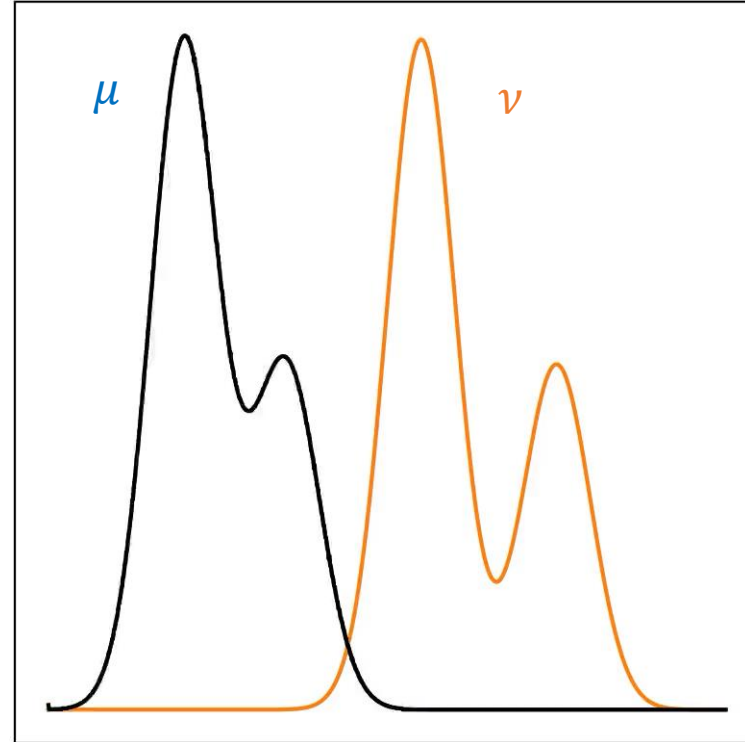
What are we looking for?

A distance **W** in the space of probability measures

$\alpha = 0.000$



$\alpha = 0.000$



Credits: Rémi Emonet

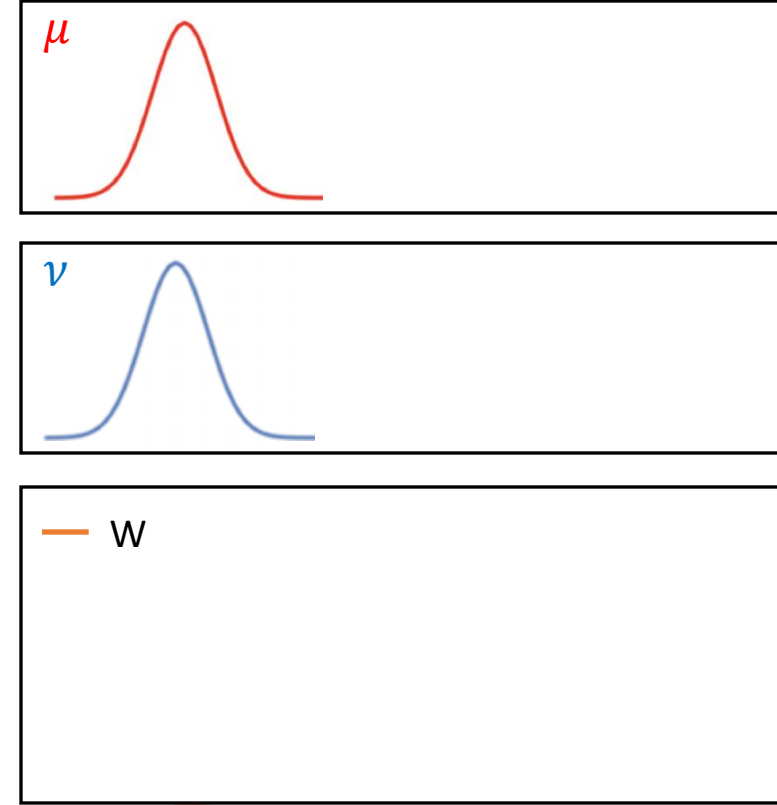
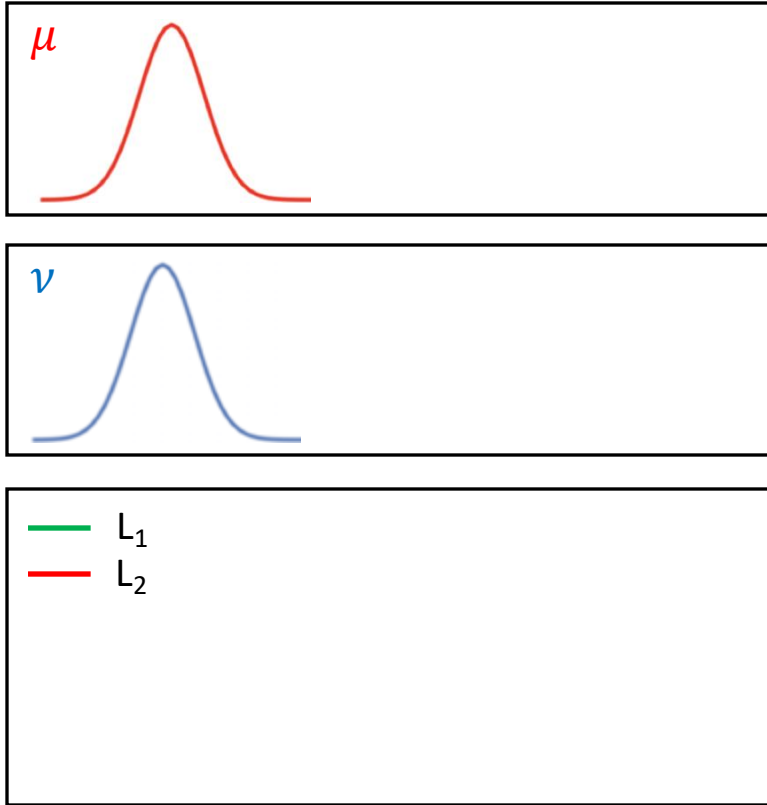
$(1 - \alpha)\mu + \alpha\nu$?

W – Interpolation

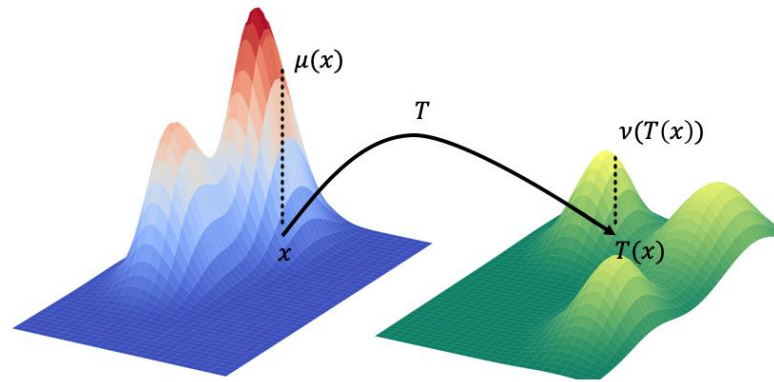


The interpolation should lead to “similar” distributions
The mass should slide along the x axis

Limitations of standard distances/divergences (ctd)



- ➔ With no overlapping support, L_1 and L_2 are constant while the distance keeps increasing
- ➔ Problem of « 0 gradient » when used in gradient descent algorithms



Need of a geometric theory that allows the comparaison of two distributions by putting a distance (**Wasserstein distance**) in the space of probabillity measures

→ **Optimal Transport**

OT has been built on the success over the last centuries of several people



Gaspard Monge
« Théorie des déblais et des remblais »
1781



Leonid Kantorovich
Nobel Prize 1975



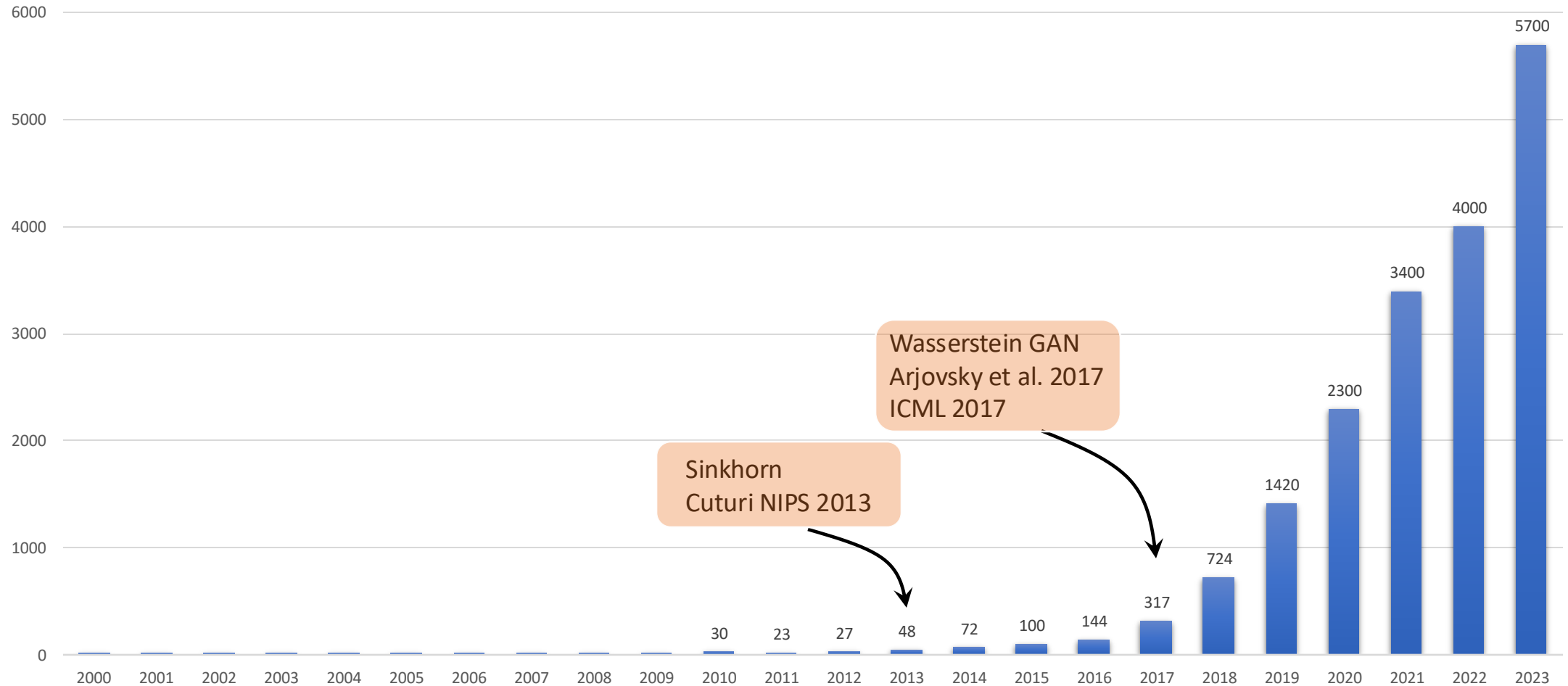
Cédric Villani
Fields Medal 2010



Alessio Figalli
Fields Medal 2018

Optimal Transport for Machine Learning

Co-OCCURRENCES OF OT and ML
in Google Scholar



OUTLINE

- Introduction
- **Monge Problem (1781)**
- **Kantorovich Problem (1942) - Wasserstein Distance**
- **Solving the OT problem – Regularized Wasserstein Distance**
- **Gromov, Fused-Gromov and Diffusion Wasserstein Distances**
- **Applications in Machine Learning and AI**

OUTLINE

- Introduction
- **Monge Problem (1781)**
- Kantorovich Problem (1942) - Wasserstein Distance
- Solving the OT problem – Regularized Wasserstein Distance
- Gromov, Fused-Gromov and Diffusion Wasserstein Distances
- Applications in Machine Learning and AI

M É M O I R E

S U R L A

T H É O R I E D E S D É B L A I S
E T D E S R E M B L A I S.

Par M. M O N G E.



Gaspard Monge
« Théorie des déblais et des remblais »
1781

When one has to bring earth from one place to another...

LORSQU'ON doit transporter une terre dans un autre, on se donne le nom de *Déblai* au volume qu'on doit transporter, & le nom de *Remblai* à l'espace qu'elles doivent occuper après le transport. Le prix du transport d'une molécule étant, toutes choses d'ailleurs égales, proportionnel à son poids & à l'espace qu'on lui fait parcourir, & par conséquent le prix du transport total devant être proportionnel à la somme des produits des molécules multipliées chacune par l'espace parcouru, il s'ensuit que le déblai & le remblai étant donnés de figure & de position, il n'est pas indifférent que telle molécule du déblai soit transportée dans tel ou tel autre endroit du remblai, mais qu'il y a une certaine distribution à faire des molécules du premier dans le second, d'après laquelle la somme de ces produits sera la moindre possible, & le prix du transport total fera un *minimum*.

C'est la solution de cette question que je me propose de donner ici. Je diviserai ce Mémoire en deux parties, dans la première je supposerai que les déblais & les remblais sont des aires contenues dans un même plan : dans le second, je supposerai que ce sont des volumes.

P R E M I È R E P A R T I E.

Du transport des aires planes sur des aires comprises dans un même plan.

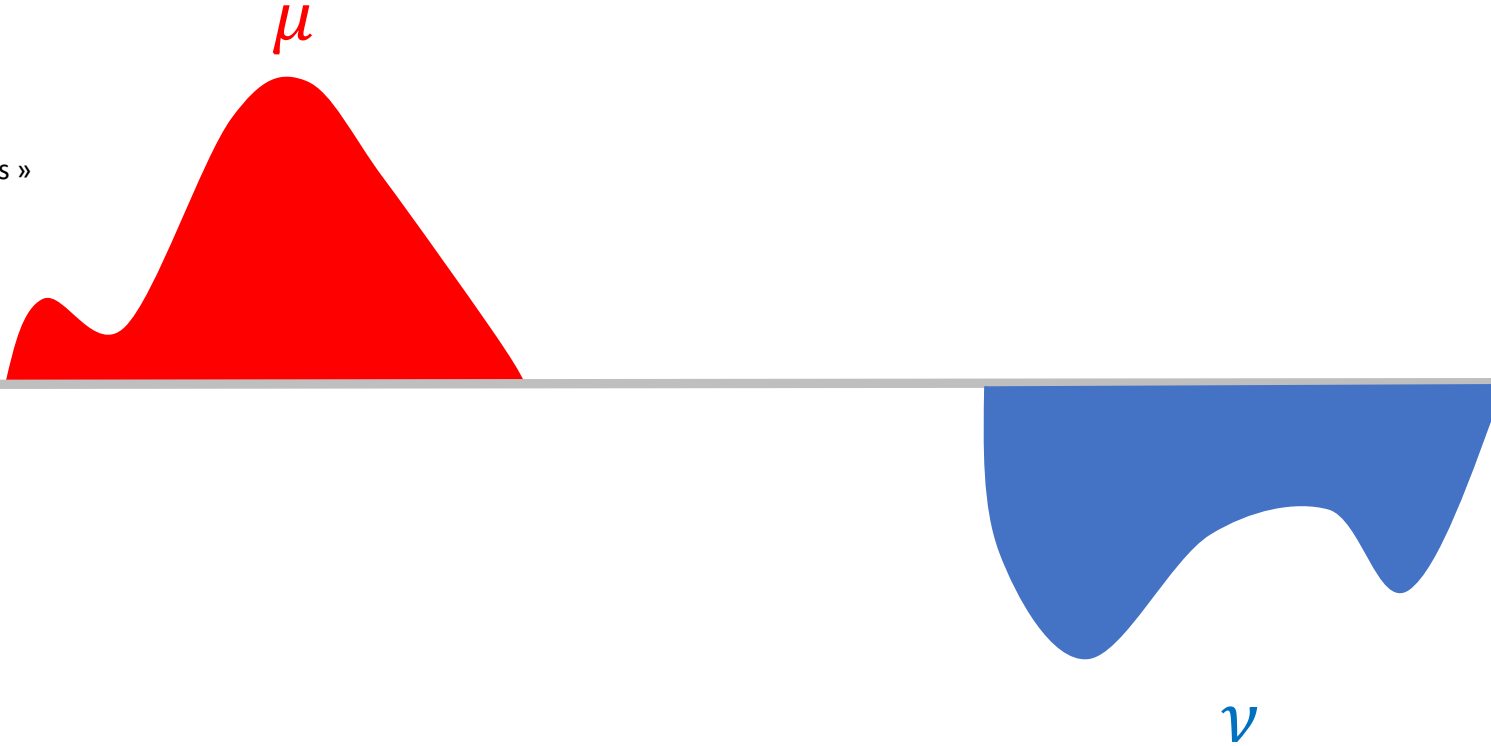
How to take earth from one place and put it in another location in an optimal way?

Monge Problem (1781)



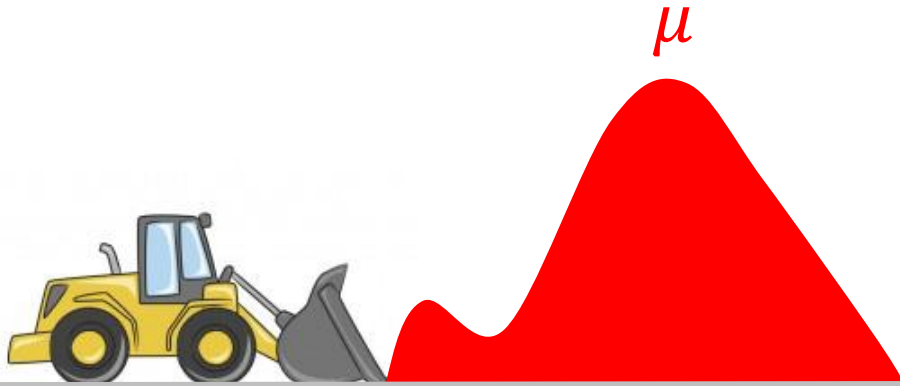
Gaspard Monge

« Théorie des déblais et des remblais »



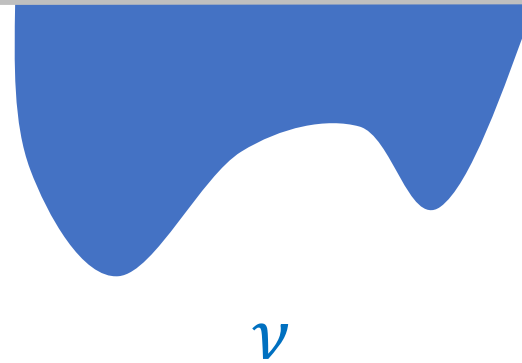
Monge Problem (1781)

If it was nowadays...



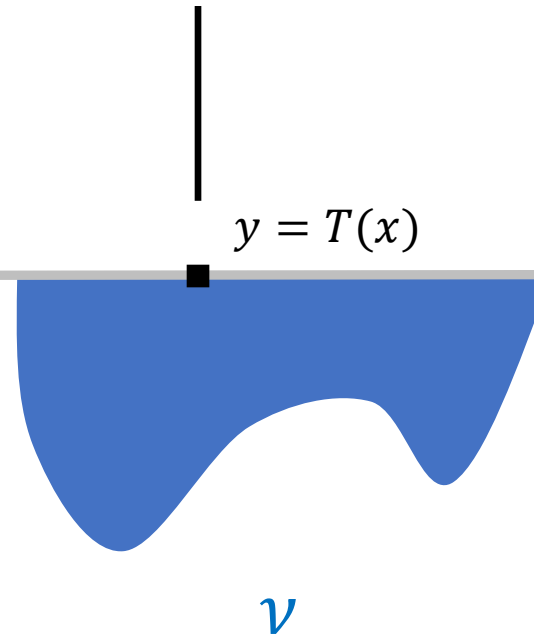
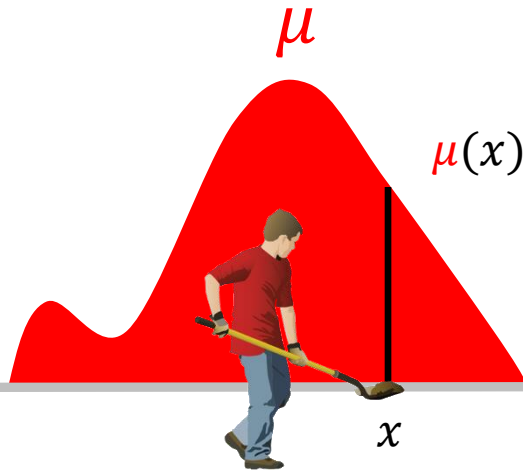
Done! No notion of optimization

The notion of effort would not really matter...

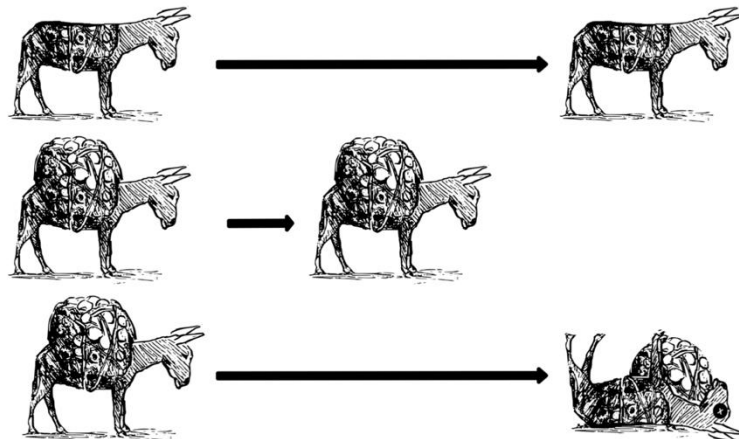


Monge Problem (1781)

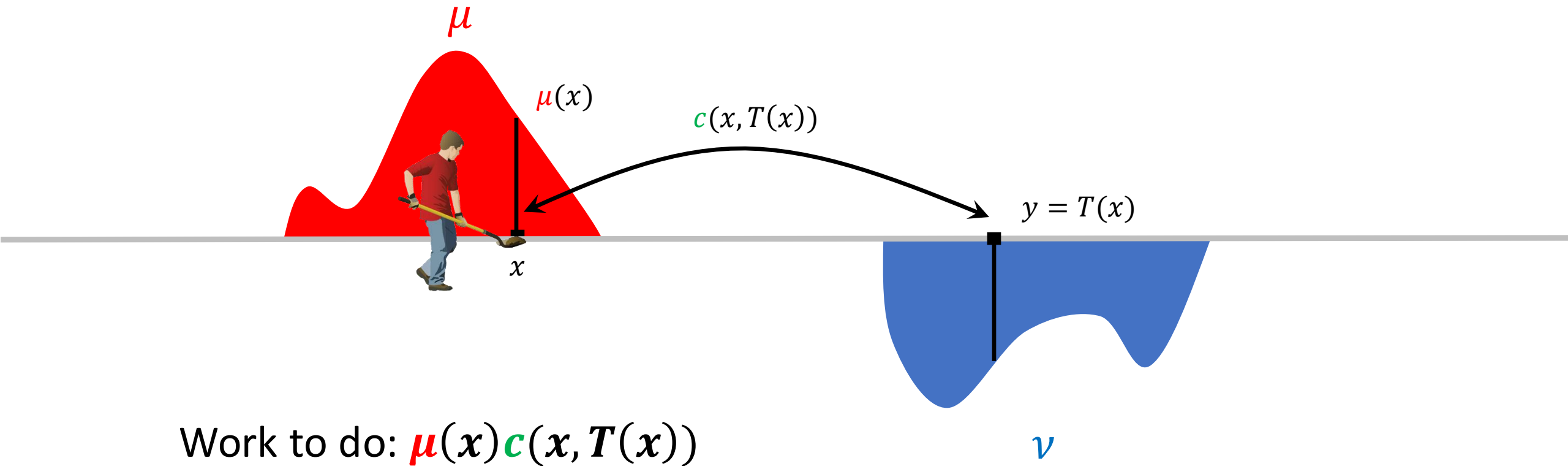
In 1781...no bulldozer



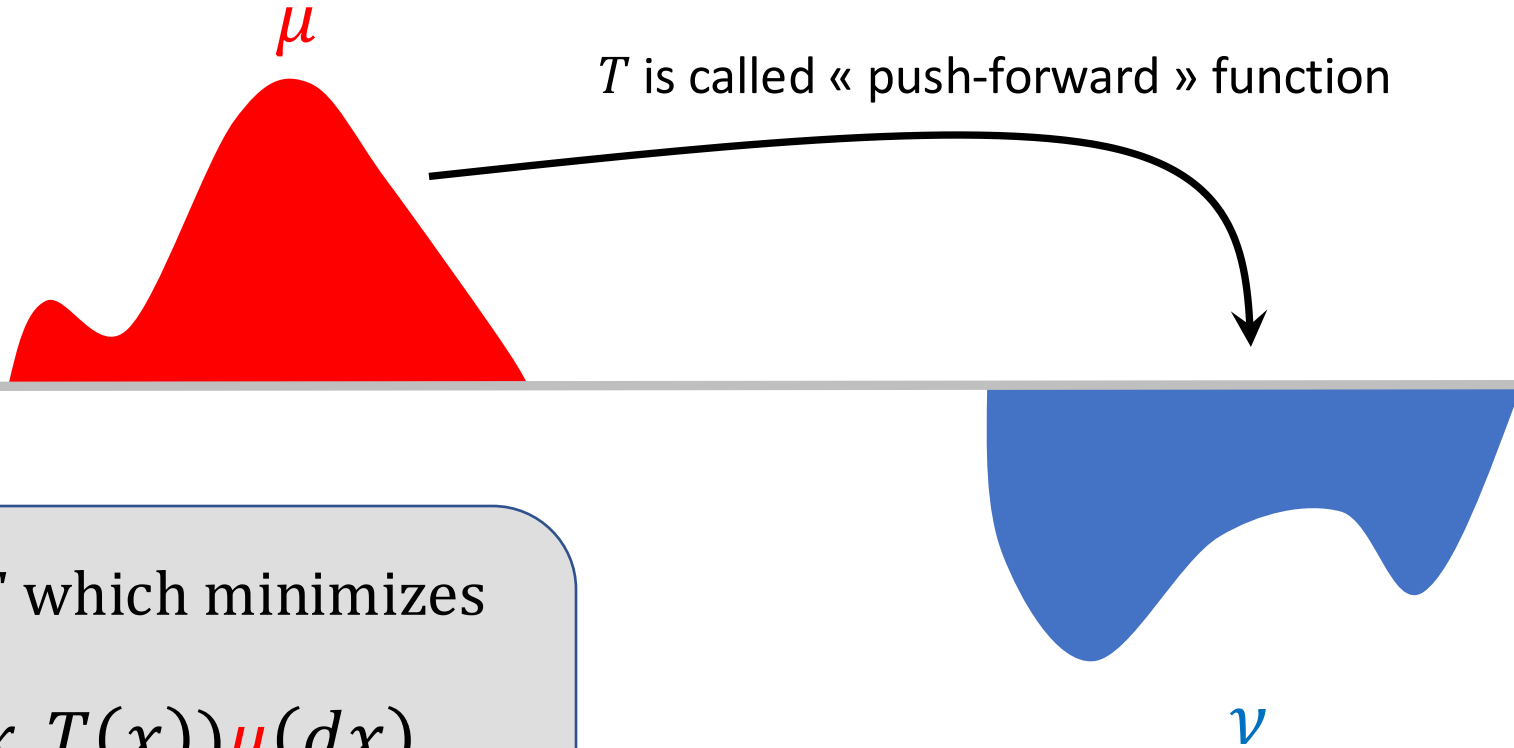
Minimizing the transportation cost: Heavy mass shouldn't be transported far!



Monge Problem (1781)



Monge Problem (1781)



Find T which minimizes

$$\int c(x, T(x)) \mu(dx)$$

such that $T_{\#}\mu = \nu$

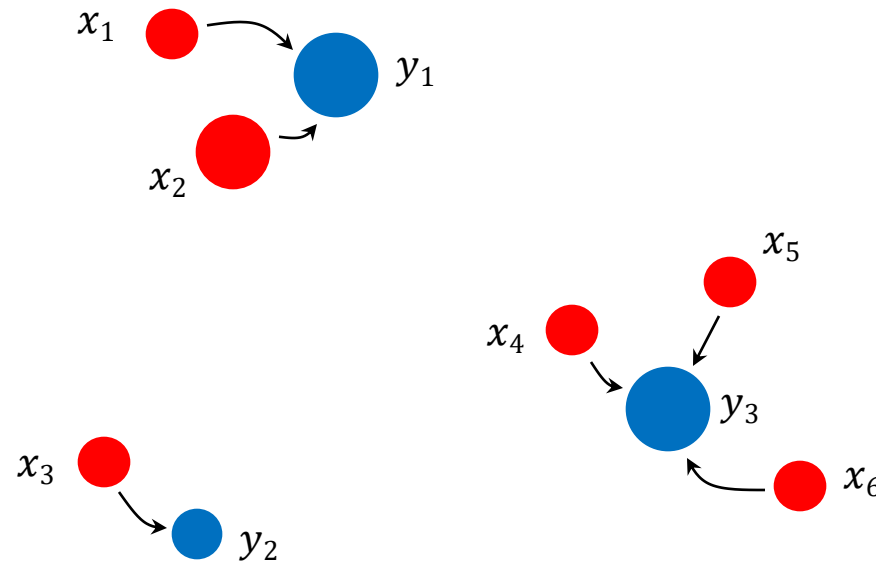
Monge Problem: Discrete Case

In ML, we often deal with point cloud distributions

Find T which minimizes

$$\sum_{x_i} c(x_i, T(x)) \mu(x_i)$$

such that $T_{\#}\mu = \nu$



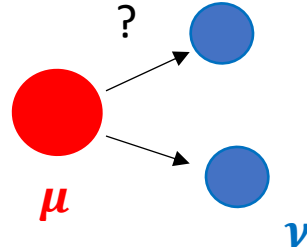
The diagram illustrates the discrete Monge problem with point cloud distributions. It shows two point clouds, μ (red) and ν (blue), each represented by a set of points within a diamond-shaped region. The points in μ are connected to the points in ν by vertical lines, representing the mapping T .

$$\mu = \sum_{i=1}^n a_i \delta_i$$
$$\nu = \sum_{j=1}^m b_j \delta_j$$

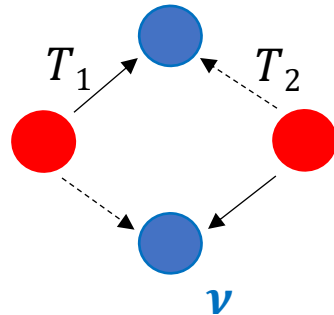
Limitations of Monge formulation

ill-posed problem (1781)

Non-existence



Non-uniqueness



Non convexity

Find T which minimizes

$$\int c(x, T(x)) \mu(dx)$$

such that $T_{\#}\mu = \nu$

With the integral...

...and non convex constraints...

Frightening looking...

[Brenier'1988] Existence Theorem

Given $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$

If $c(x, y) = \|x - y\|^2$ and μ and ν are absolutely continuous, there exists a unique optimal $T = \nabla u$ with u convex.

It took many years to make progress on it...

OUTLINE

- Introduction
- Monge Problem (1781)
- **Kantorovich Problem (1942) - Wasserstein Distance**
- Solving the OT problem – Regularized Wasserstein Distance
- Gromov, Fused-Gromov and Diffusion Wasserstein Distances
- Applications in Machine Learning and AI

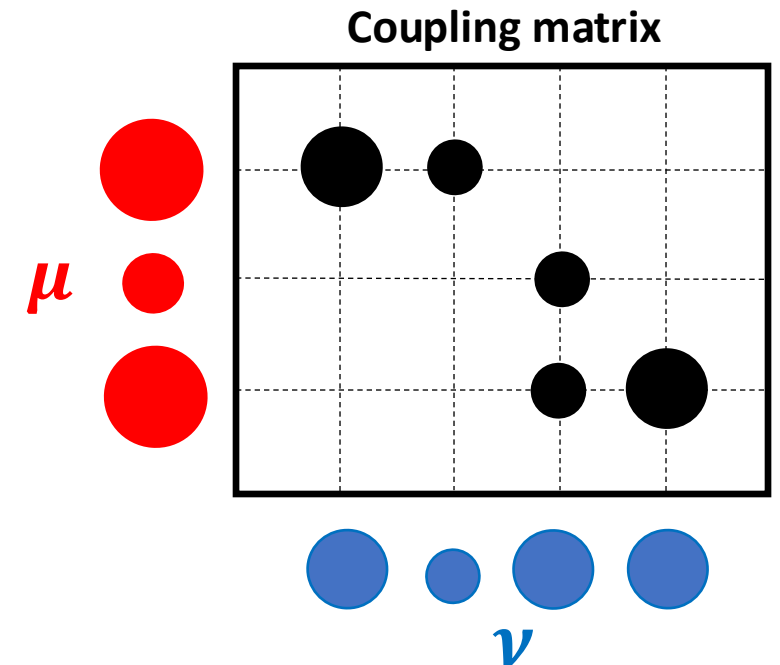
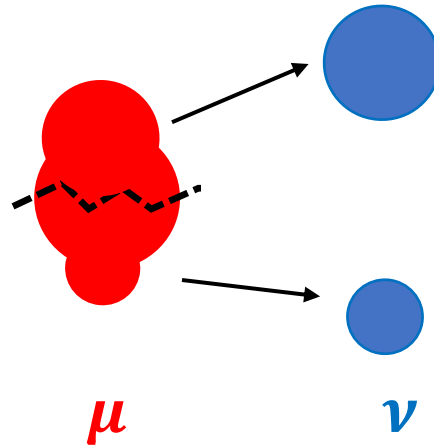
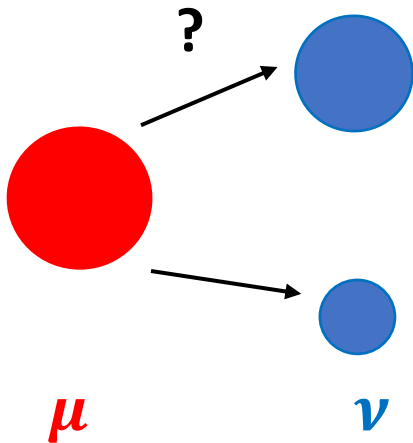
Kantorovich Problem (1942)



Leonid Kantorovich
Nobel Prize 1975

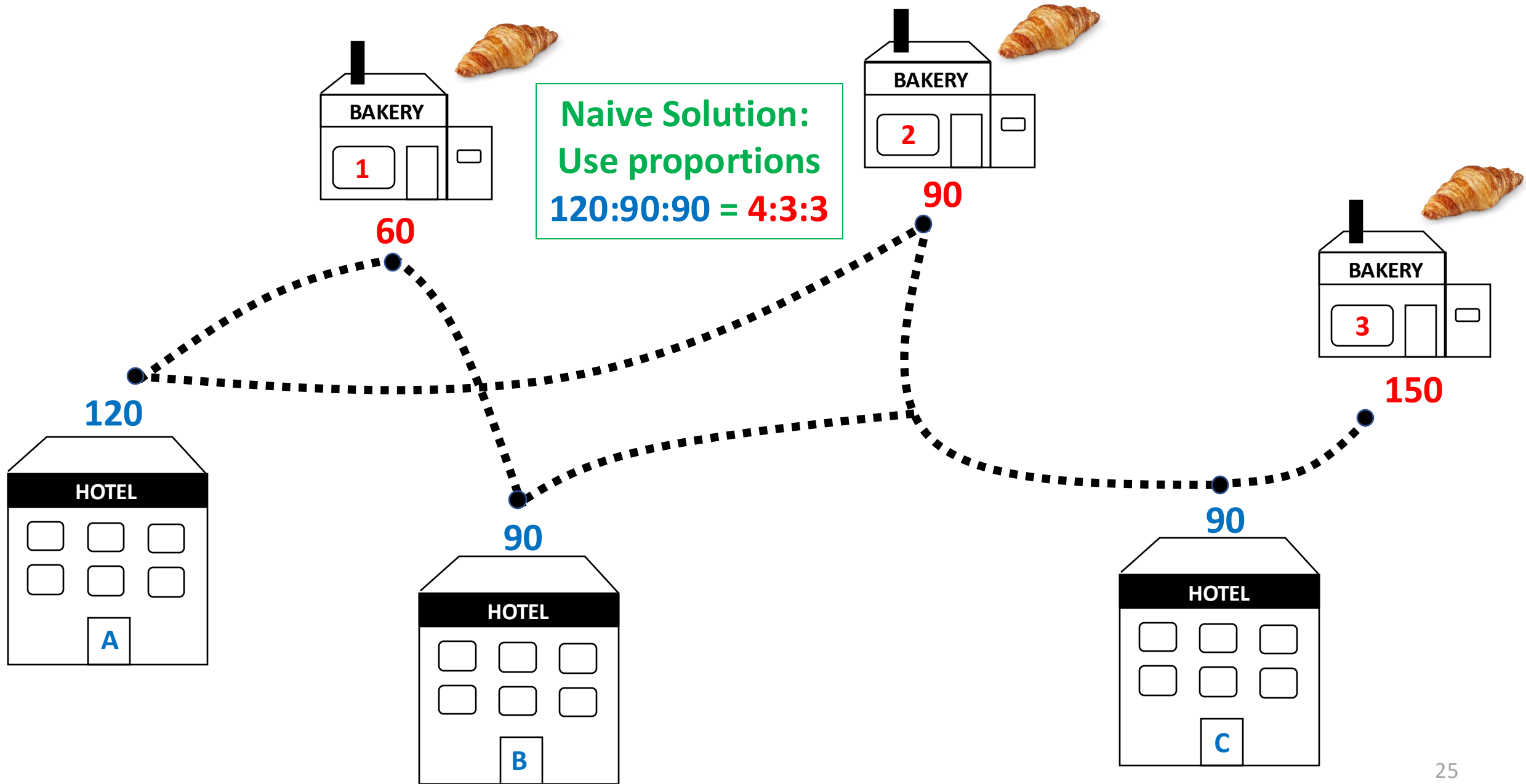
Kantorovich came to rescue by introducing a relaxed version of the problem yielding much better mathematical properties

It is allowed to split the mass of probability $\longrightarrow$ joint probabilistic coupling

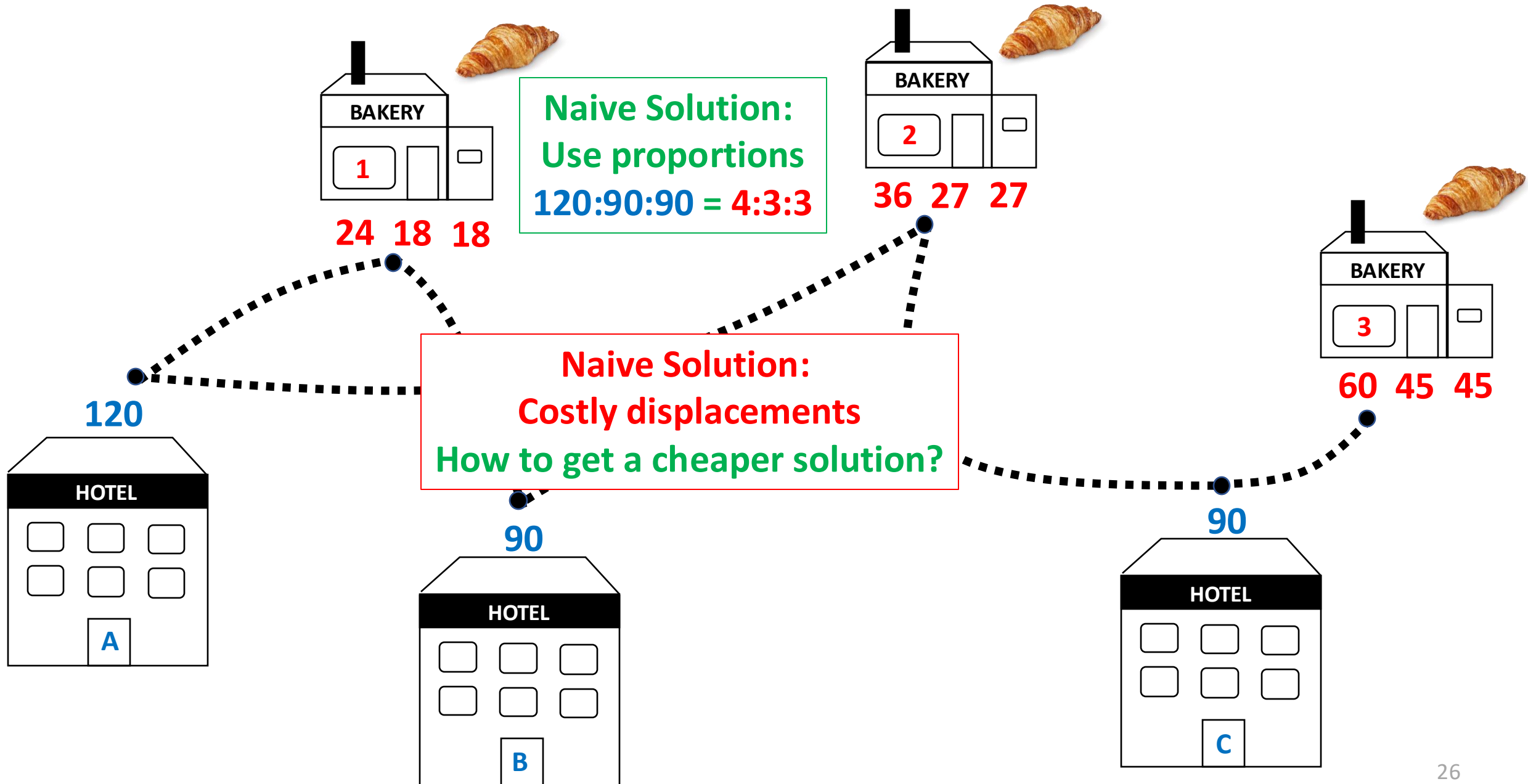


Possibility to have for one x different images y

Kantorovich Problem: illustration



Kantorovich Problem: illustration



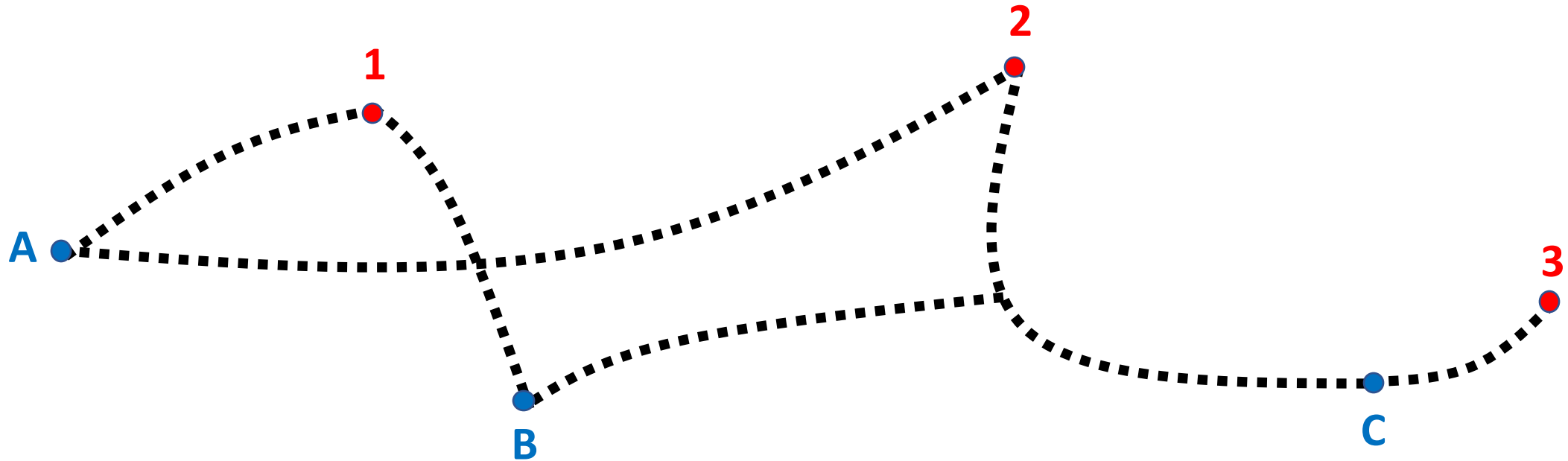
Kantorovich Problem

Transportation matrix γ

60	?	?	?
90	?	?	?
150	?	?	?
	120	90	90

Distance matrix D

1	d_{1A}	d_{1B}	d_{1C}
2	d_{2A}	d_{2B}	d_{2C}
3	d_{3A}	d_{3B}	d_{3C}
	A	B	C



Kantorovich Problem

Transportation matrix γ

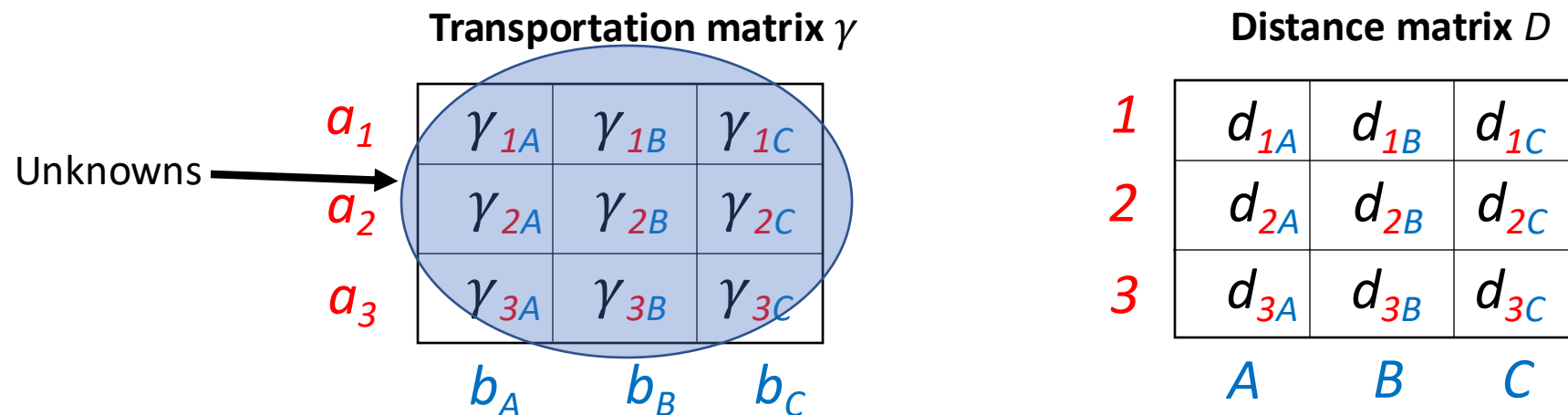
60	?	?	?
90	?	?	?
150	?	?	?
	120	90	90

Distance matrix D

1	d_{1A}	d_{1B}	d_{1C}
2	d_{2A}	d_{2B}	d_{2C}
3	d_{3A}	d_{3B}	d_{3C}
	A	B	C

The problem is described by counts and a cost/distance matrix

Kantorovich Problem



In the discrete case, the Kantorovich formulation seeks for the following probabilistic coupling:

$$\gamma^* = \min_{\gamma} \sum_{i=1}^n \sum_{j=1}^m \gamma_{ij} d_{ij} = \min_{\gamma} \langle \gamma, D \rangle_F$$

Frobenius inner product

Linear objective function

+

At least,
 $\gamma = ab^T$
 is a (naive) solution

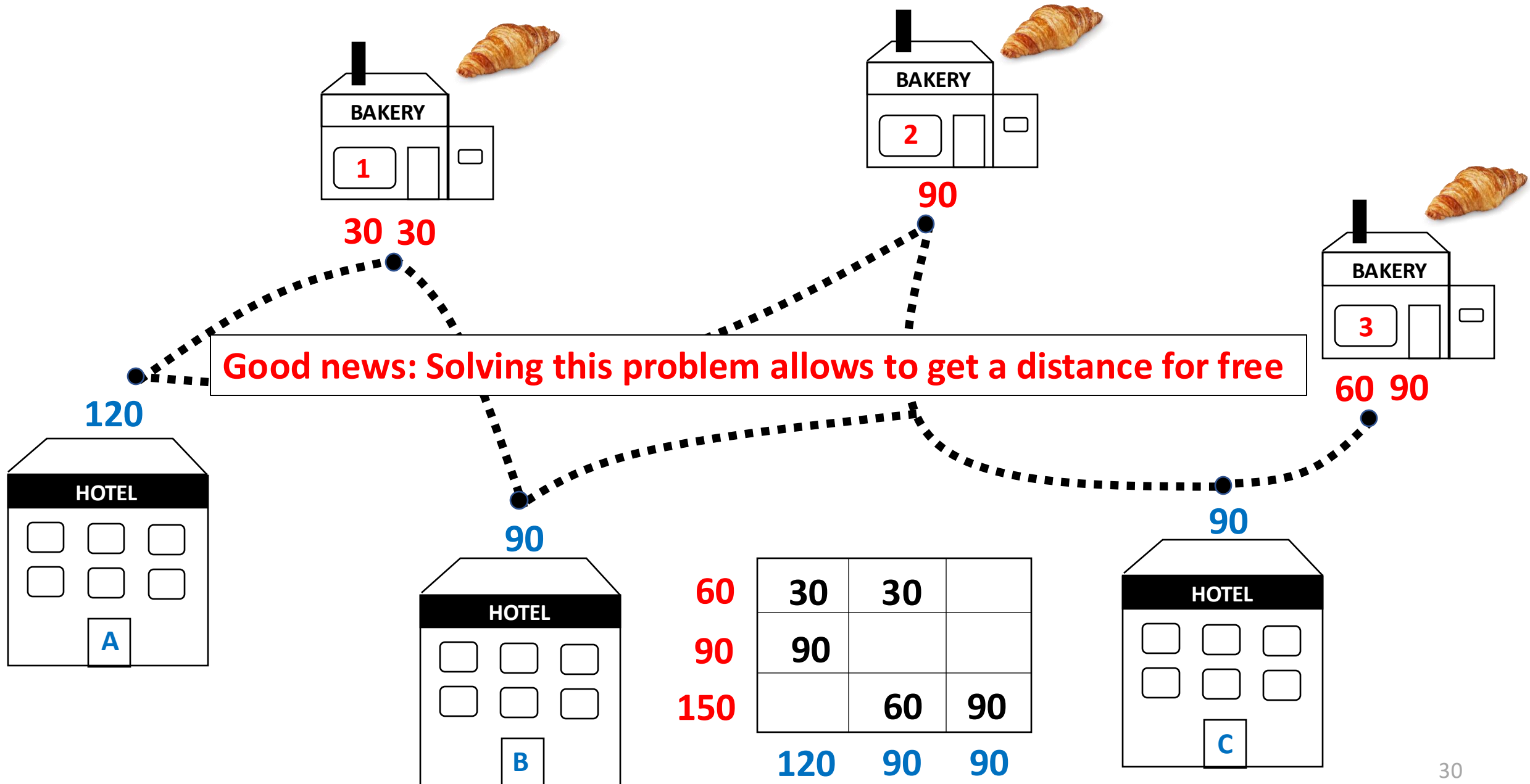
s.t. $\gamma \in U_{ab} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = a; \gamma^T \mathbf{1}_n = b\}$

Linear constraints

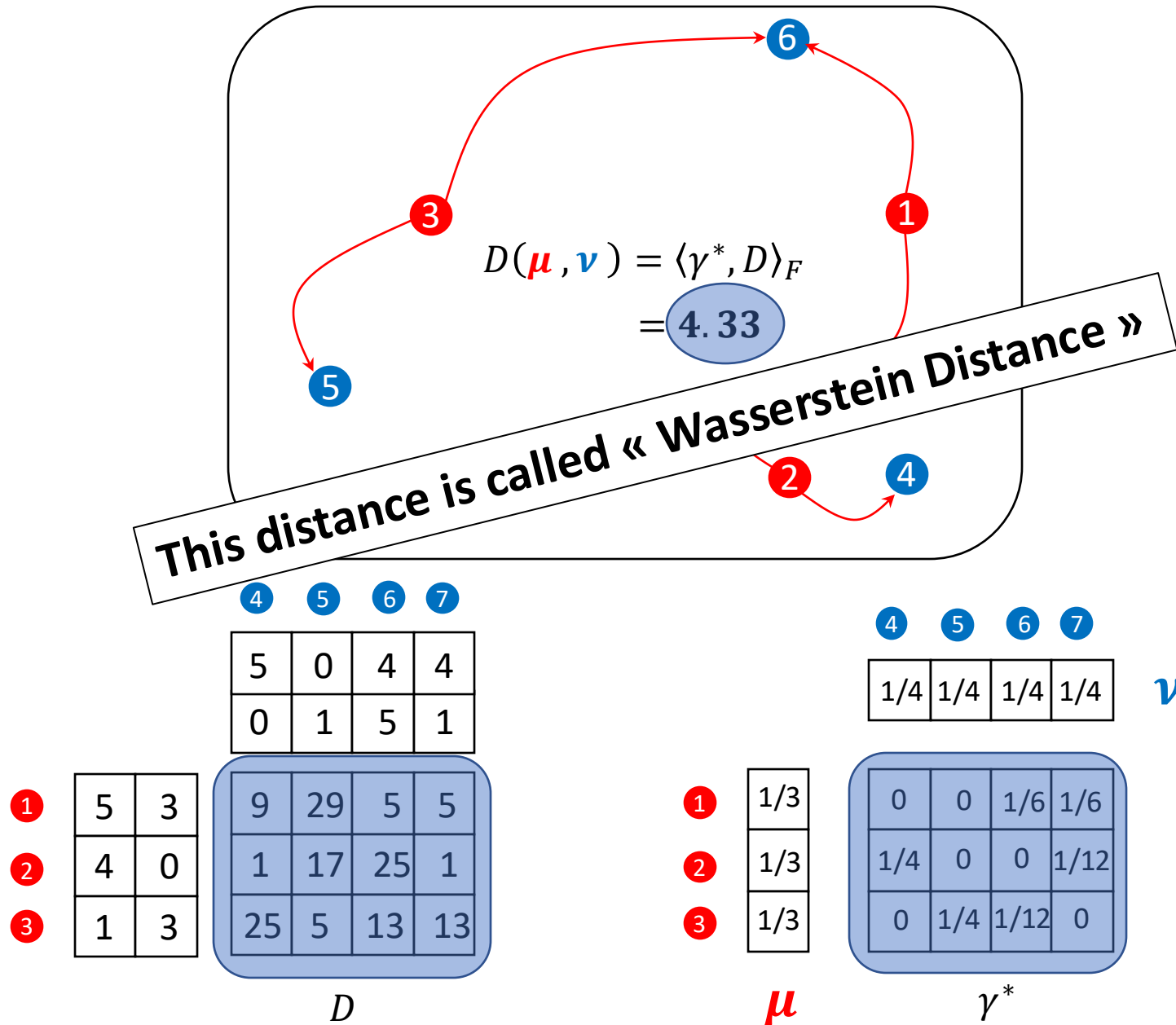
=

Linear program that always has a solution

Kantorovich Problem: illustration



From the coupling matrix to a distance between μ and ν



Wasserstein Distance

The p-Wasserstein distance between μ and ν is defined as follows:

$$W_p^p(\mu, \nu) = \min_{\gamma \in U(a, b)} \langle \gamma, D \rangle_F$$

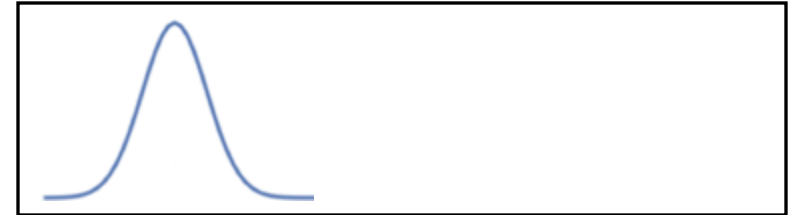
$$\text{where } D_{xy} = \|x - y\|^p$$

- $W_p^p(\mu, \nu)$ is a distance between probability measures
- It satisfies the properties of an actual distance
- $W_1^1(\mu, \nu)$ is a.k.a. Earth Mover's distance (EMD)
- Does not require overlapping support between μ and ν

μ

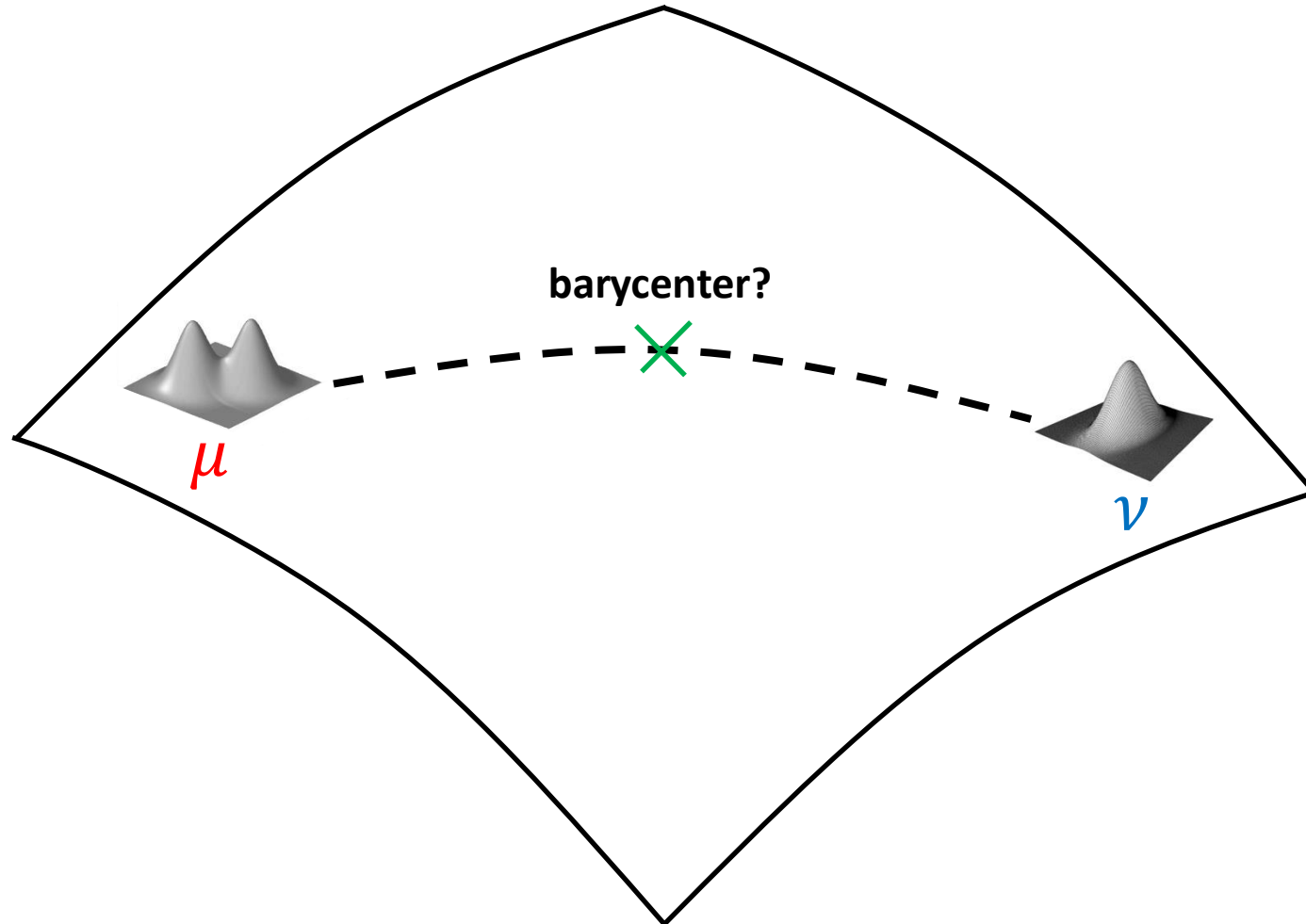


ν



— W_2^2
— L_2

The geometry behind the Wasserstein Distance

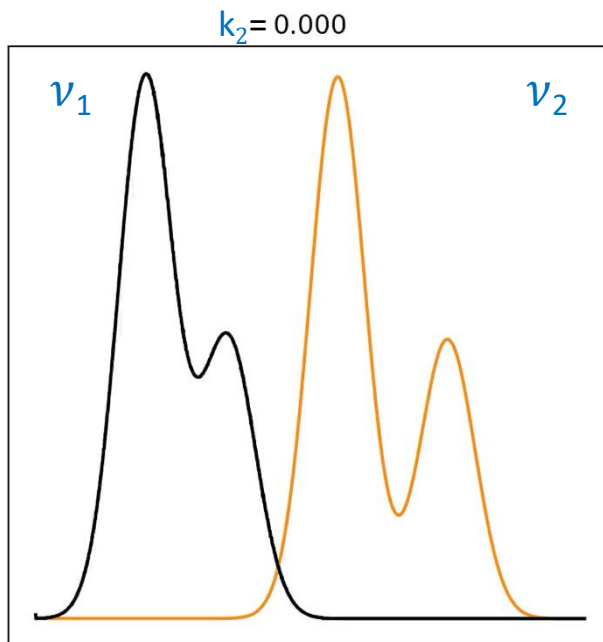


Wasserstein Barycenter

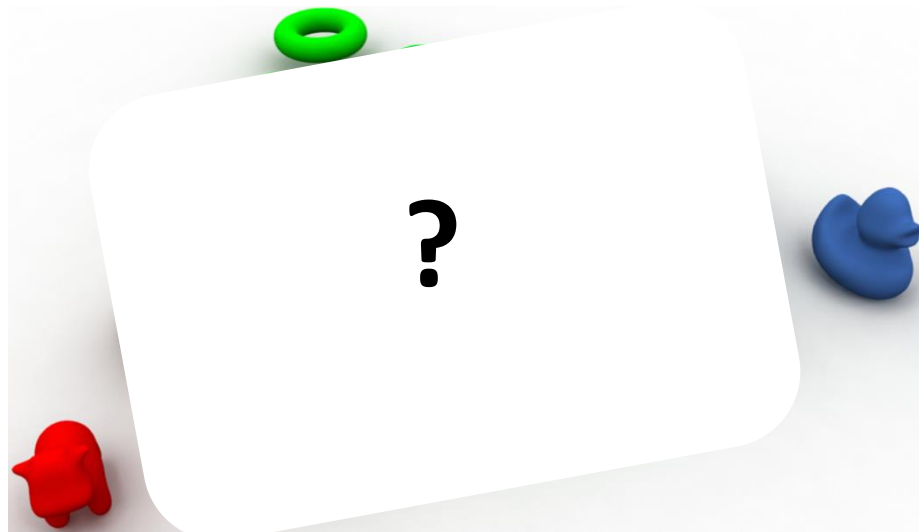
The Wasserstein barycenter B of a set of N probability measures $\nu_1, \dots, \nu_N$ is defined as follows:

$$\min_B \sum_{i=1}^N k_i W_p^p(B, \nu_i)$$

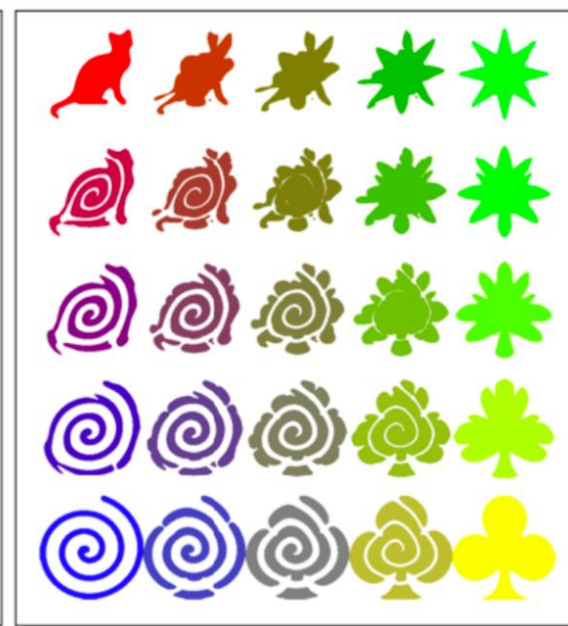
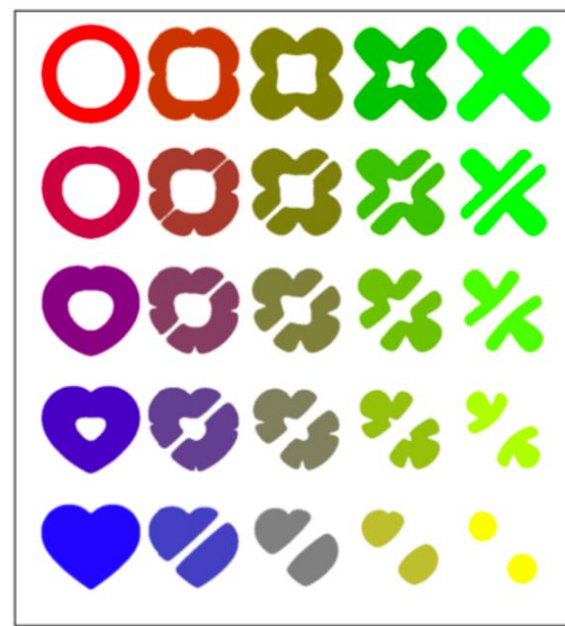
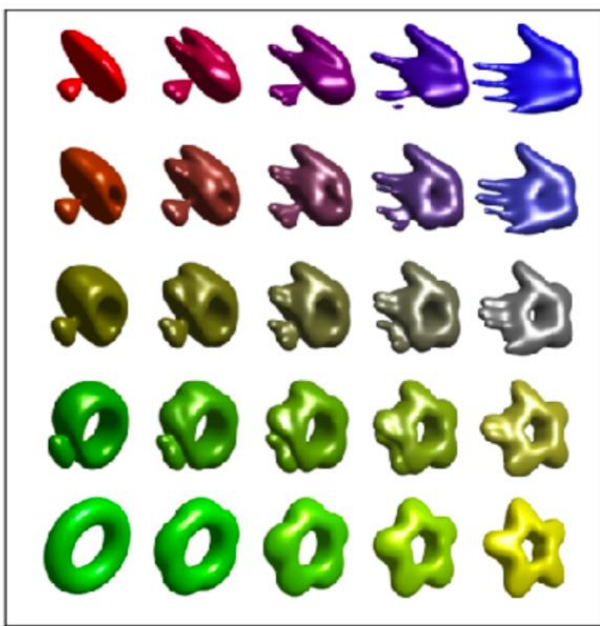
$(1 - k_2)\nu_1 + k_2\nu_2?$



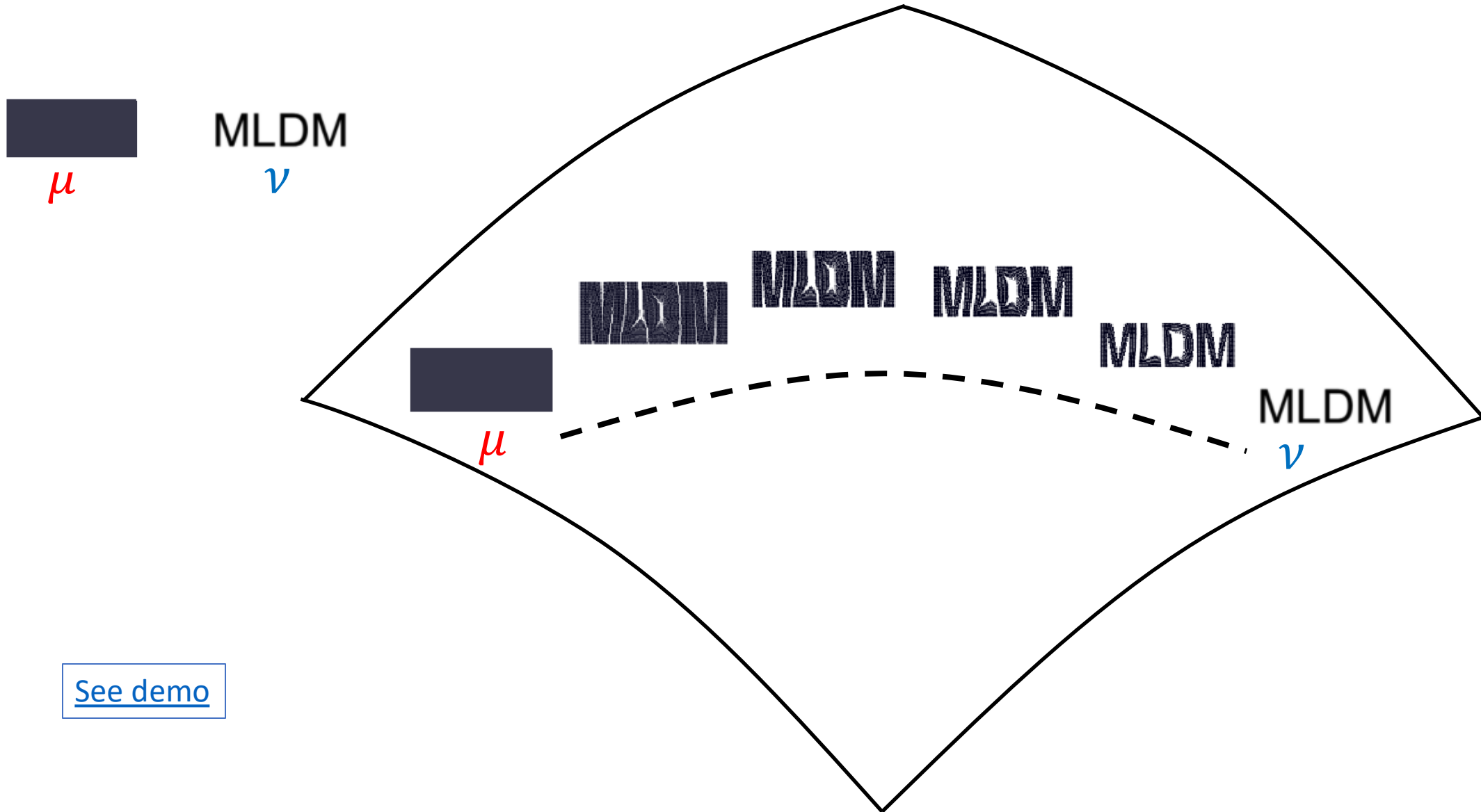
Credits: R. Emonet



Wasserstein Barycenters



Wasserstein Barycenters

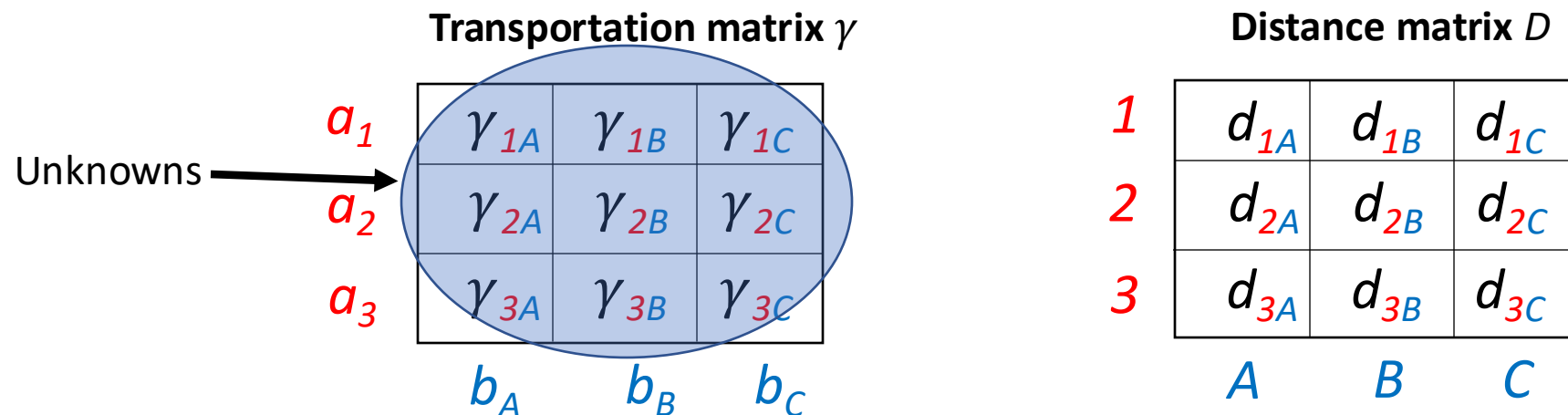


[See demo](#)

OUTLINE

- Introduction
- Monge Problem (1781)
- Kantorovich Problem (1942) - Wasserstein Distance
- **Solving the OT problem – Regularized Wasserstein Distance**
- Gromov, Fused-Gromov and Diffusion Wasserstein Distances
- Applications in Machine Learning and AI

Kantorovich Problem



In the discrete case, the Kantorovich formulation seeks for the following probabilistic coupling:

$$\gamma^* = \min_{\gamma} \sum_{i=1}^n \sum_{j=1}^m \gamma_{ij} d_{ij} = \min_{\gamma} \langle \gamma, D \rangle_F$$

Frobenius inner product

Linear objective function

+

At least,
 $\gamma = ab^T$
 is a (naive) solution

s.t. $\gamma \in U_{ab} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = a; \gamma^T \mathbf{1}_n = b\}$

Linear constraints

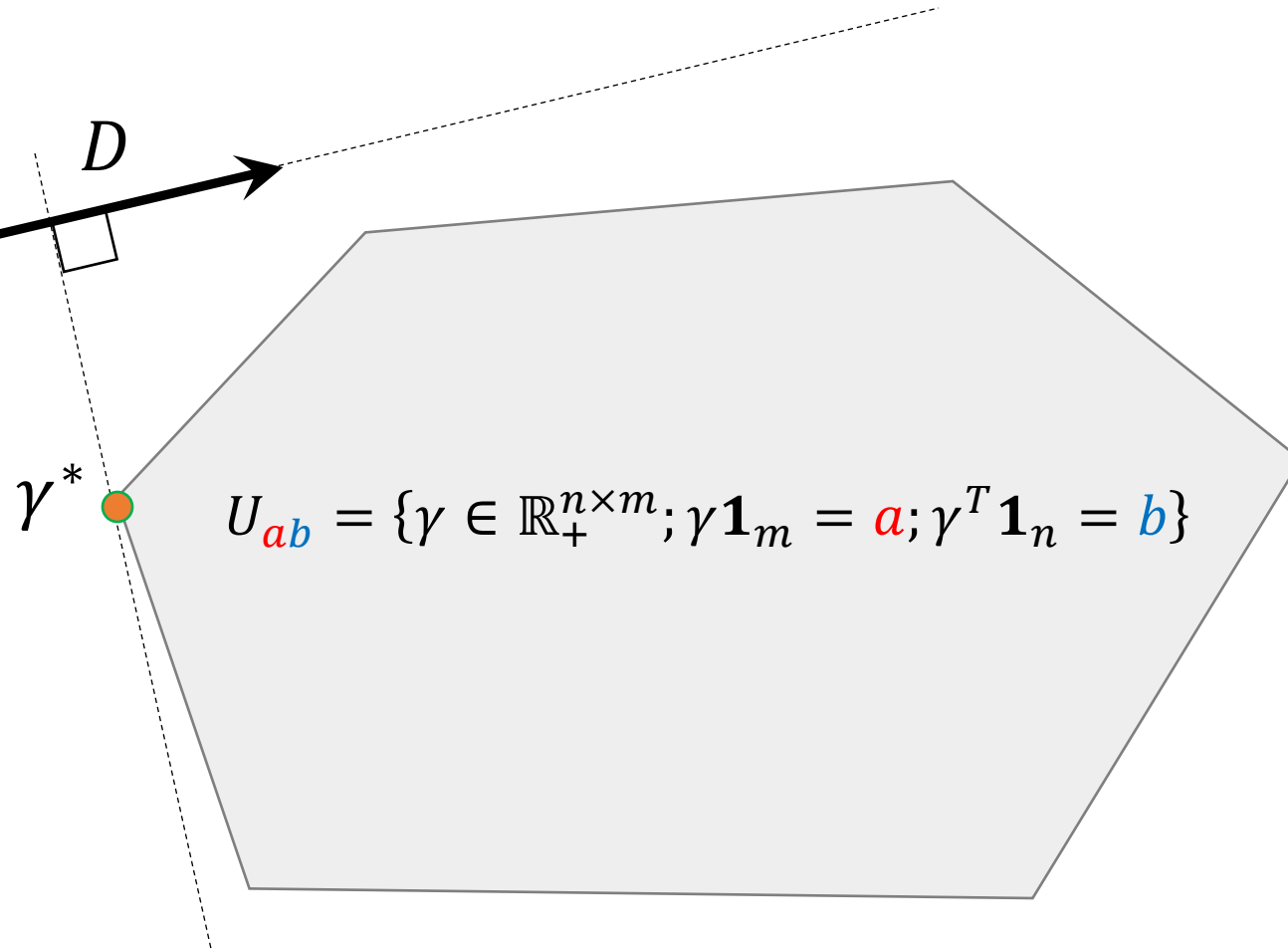
=

Linear program that always has a solution

Solving the Kantorovich Problem

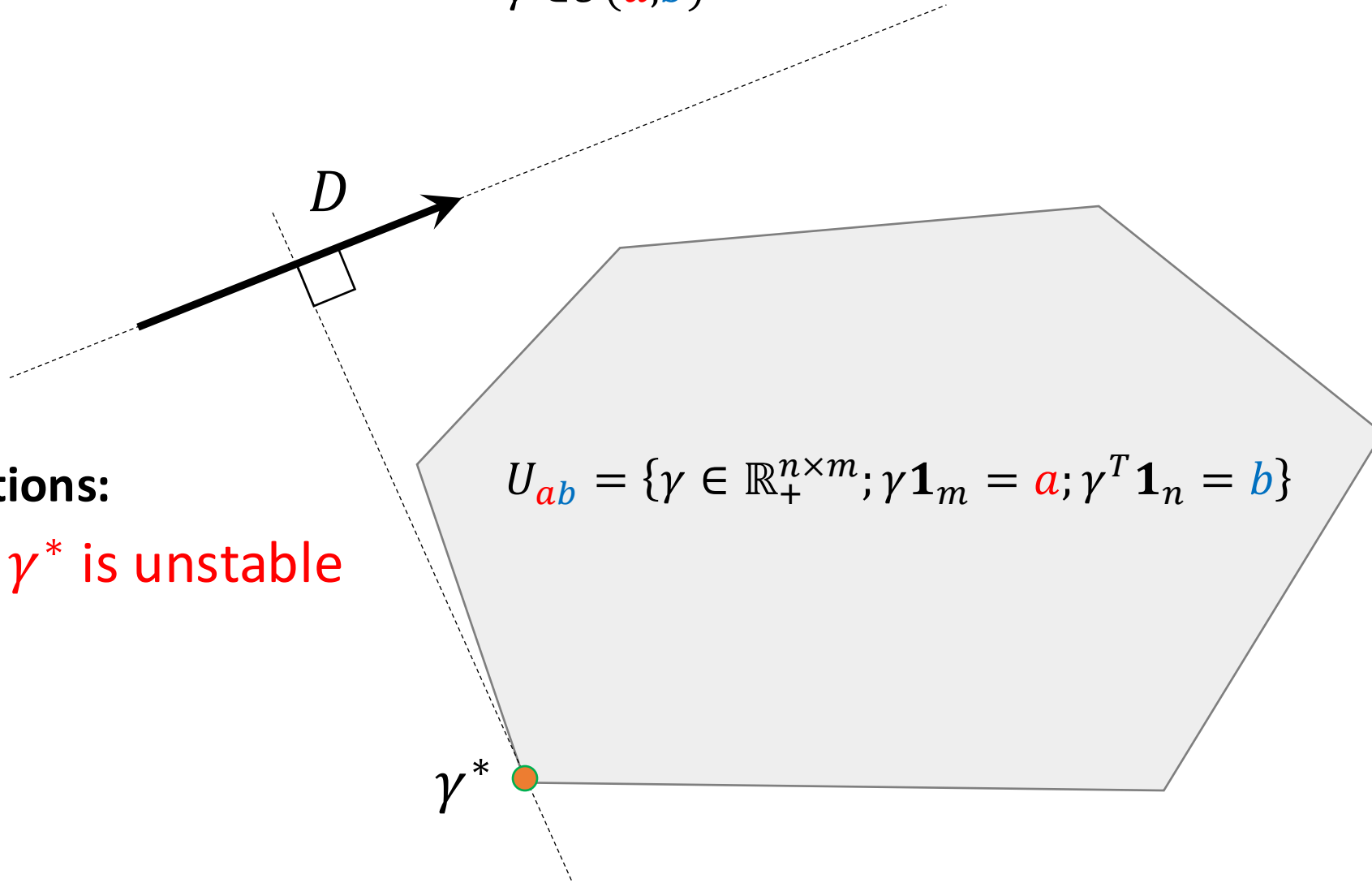
$$\min_{\gamma \in U(\textcolor{red}{a}, \textcolor{blue}{b})} \langle \gamma, D \rangle_F$$

3 limitations:



Solving the Kantorovich Problem

$$\min_{\gamma \in U(\textcolor{red}{a}, \textcolor{blue}{b})} \langle \gamma, D \rangle_F$$



3 limitations:

(i) γ^* is unstable

Solving the Kantorovich Problem

$$\min_{\gamma \in U(\mathbf{a}, \mathbf{b})} \langle \gamma, D \rangle_F + \lambda E(\gamma)$$

Regularization

How to overcome these limitations?

3 limitations:

$\{\gamma^*\}$

$$U_{\mathbf{a}\mathbf{b}} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = \mathbf{a}; \gamma^T \mathbf{1}_n = \mathbf{b}\}$$

- (i) γ^* is unstable
- (ii) not always unique
- (iii) costly

$O(N^3 \log N)$
(using min cost flow solver)

Regularized Wasserstein Distance

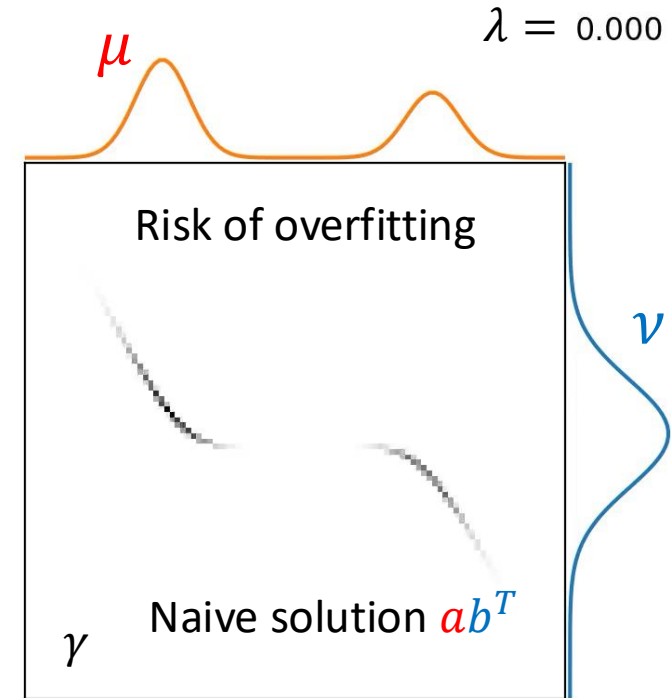
Entropic Regularization

The p-Wasserstein distance between μ and ν is defined as follows, $\lambda \geq 0$:

$$W_\lambda(\mu, \nu) = \min_{\gamma \in U(a, b)} \langle \gamma, D \rangle_F - \lambda E(\gamma)$$

where

$$E(\gamma) = - \sum_{i,j=1}^{nm} \gamma_{ij} (\log \gamma_{ij} - 1)$$



Crédits: Rémi Emonet

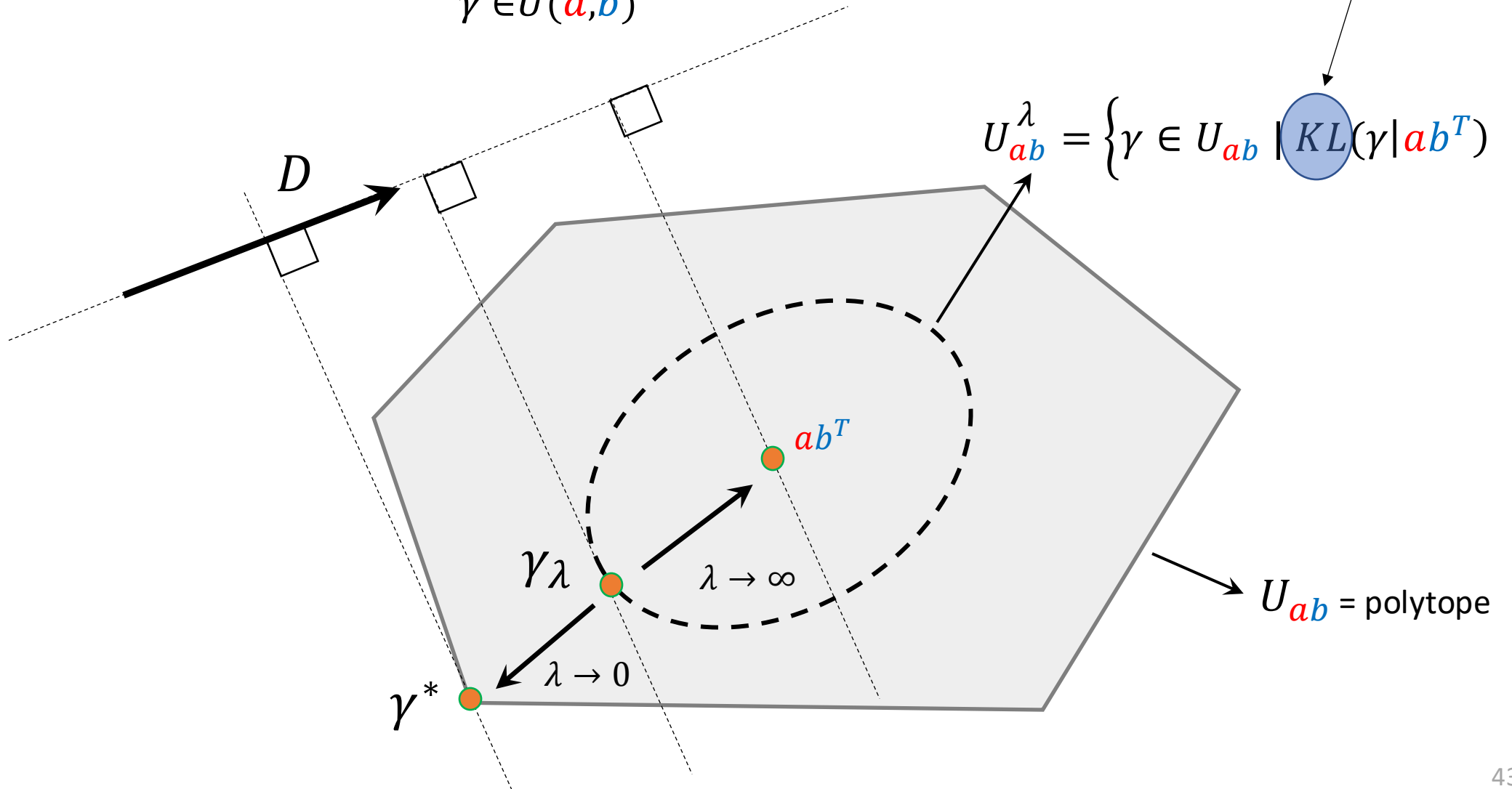
Uniqueness of γ^* thanks to the strong convexity of the entropy

Regularized OT Problem

$$\gamma_\lambda = \operatorname{argmin}_{\gamma \in U(\mathbf{a}, \mathbf{b})} \langle \gamma, D \rangle_F + \lambda E(\gamma)$$

Link with the
Kullback-Leibler Divergence

$$U_{\mathbf{a}\mathbf{b}}^\lambda = \left\{ \gamma \in U_{\mathbf{a}\mathbf{b}} \mid \text{KL}(\gamma \mid \mathbf{a}\mathbf{b}^T) \leq \frac{1}{\lambda} \right\}$$



Link between the KL and the Entropy-regularized Wasserstein Distance

Kullback-Leibler Divergence between two discrete distributions P and Q :

$$KL(P|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

Entropy-regularized Wasserstein Distance:

$$W_\lambda(\mu, \nu) = \min_{\gamma \in U(a, b)} \langle \gamma, D \rangle_F - \lambda E(\gamma)$$

$$= \min_{\gamma} \sum_{ij} \gamma_{ij} D_{ij} + \lambda \gamma_{ij} (\log \gamma_{ij} - 1)$$

$$= \lambda \min_{\gamma} \sum_{ij} \gamma_{ij} \left(\frac{D_{ij}}{\lambda} + \log \gamma_{ij} \right) - 1$$

$$= \lambda \min_{\gamma} \sum_{ij} \gamma_{ij} \left(\log \exp \frac{D_{ij}}{\lambda} + \log \gamma_{ij} \right) - 1 = \lambda \min_{\gamma} \sum_{ij} \gamma_{ij} \left(\log \frac{\gamma_{ij}}{\exp(-D_{ij}/\lambda)} \right) - 1$$

$$= \lambda \min_{\gamma} KL(\gamma|K) - 1 \quad \text{with } K = \exp(-\frac{D}{\lambda})$$

Is there an efficient algorithm
for solving the regularized OT problem?



Solving the regularized problem **in the primal**

$$\begin{aligned}\gamma_\lambda &= \operatorname{argmin}_{\gamma \in U(\mathbf{a}, \mathbf{b})} \langle \gamma, D \rangle_F - \lambda E(\gamma) \\ &= \operatorname{argmin}_{\gamma \in U(\mathbf{a}, \mathbf{b})} \sum_{ij} \gamma_{ij} D_{ij} + \lambda \gamma_{ij} (\log \gamma_{ij} - 1) \\ U_{\mathbf{a}\mathbf{b}} &= \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = \mathbf{a}; \gamma^T \mathbf{1}_n = \mathbf{b}\}\end{aligned}$$

Consider the Lagrangian of this constrained optimization problem:

$$\mathcal{L}(\gamma, \alpha, \beta) = \sum_{ij} \gamma_{ij} D_{ij} + \lambda \gamma_{ij} (\log \gamma_{ij} - 1) - \alpha^T (\gamma \mathbf{1}_m - \mathbf{a}) - \beta^T (\gamma^T \mathbf{1}_n - \mathbf{b})$$

$$\frac{\partial \mathcal{L}(\gamma, \alpha, \beta)}{\partial \gamma_{ij}} = D_{ij} + \lambda \log \gamma_{ij} - \alpha_i - \beta_j$$

$$\frac{\partial \mathcal{L}(\gamma, \alpha, \beta)}{\partial \gamma_{ij}} = 0 \Leftrightarrow \gamma_{ij} = \underbrace{\exp\left(\frac{\alpha_i}{\lambda}\right)}_{u_i} \underbrace{\exp\left(-\frac{D_{ij}}{\lambda}\right)}_{K_{ij}} \underbrace{\exp\left(\frac{\beta_j}{\lambda}\right)}_{v_j} = \boxed{u_i K_{ij} v_j} \rightarrow \boxed{\gamma = \operatorname{diag}(\mathbf{u}) K \operatorname{diag}(\mathbf{v})}$$

one entry of γ
the entire γ

Let's plug γ in the constraints

Solving the regularized problem

$$U_{ab} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = a; \gamma^T \mathbf{1}_n = b\}$$

By plugging $\gamma = \text{diag}(u)K\text{diag}(v)$ in the constraints, we get:

$$\gamma \in U(a, b) \Leftrightarrow \begin{cases} \gamma \mathbf{1}_m & = a \\ \gamma^T \mathbf{1}_n & = b \end{cases}$$

Solving the regularized problem

$$\gamma \in U(\textcolor{red}{a}, \textcolor{blue}{b}) \Leftrightarrow \begin{cases} \mathbf{diag}(\textcolor{red}{u})K \mathbf{diag}(\textcolor{blue}{v})\mathbf{1}_m & = \textcolor{red}{a} \\ \mathbf{diag}(\textcolor{blue}{v})K^T \mathbf{diag}(\textcolor{red}{u})\mathbf{1}_n & = \textcolor{blue}{b} \end{cases}$$

Solving the regularized problem

$$\gamma \in U(a, b) \Leftrightarrow \begin{cases} \mathbf{diag}(u)Kv & = a \\ \mathbf{diag}(v)K^T u & = b \end{cases}$$

Solving the regularized problem

$$\gamma \in U(a, b) \Leftrightarrow \begin{cases} \textcolor{red}{u} \odot K \textcolor{blue}{v} & = \textcolor{red}{a} \\ \textcolor{blue}{v} \odot K^T \textcolor{red}{u} & = \textcolor{blue}{b} \end{cases}$$

with $\odot$ the element-wise product

Solving the regularized problem

Sinkhorn's Algorithm

Reminder: $u_i = \exp\left(\frac{\alpha_i}{\lambda}\right)$ and $v_j = \exp\left(\frac{\beta_j}{\lambda}\right)$

$$\gamma \in U(a, b) \Leftrightarrow \begin{cases} u = a/Kv \\ v = b/K^T u \end{cases}$$

System of non linear equations

①
②

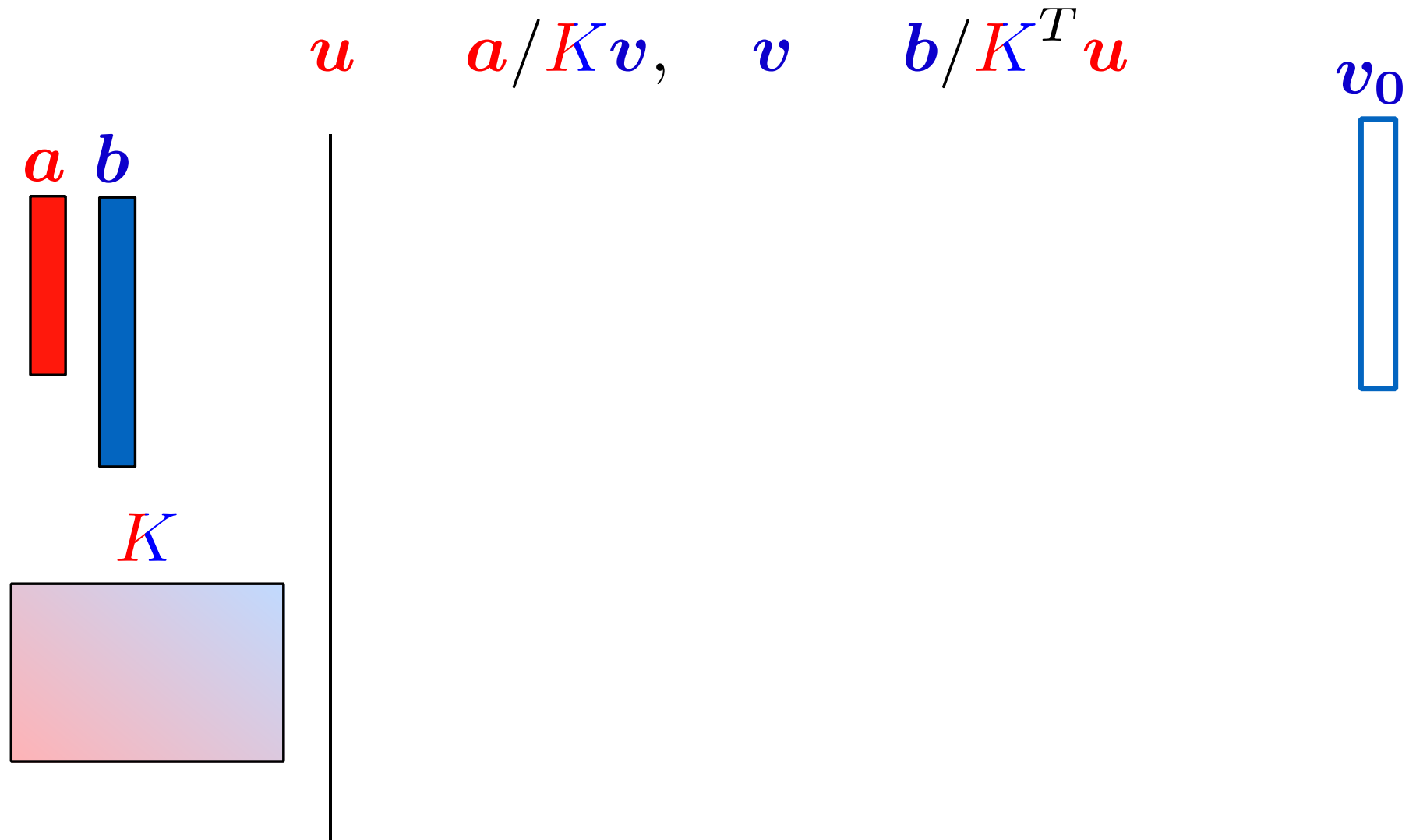
(fixed-point iterations)

Sinkhorn's Algorithm
(Cuturi 2013)

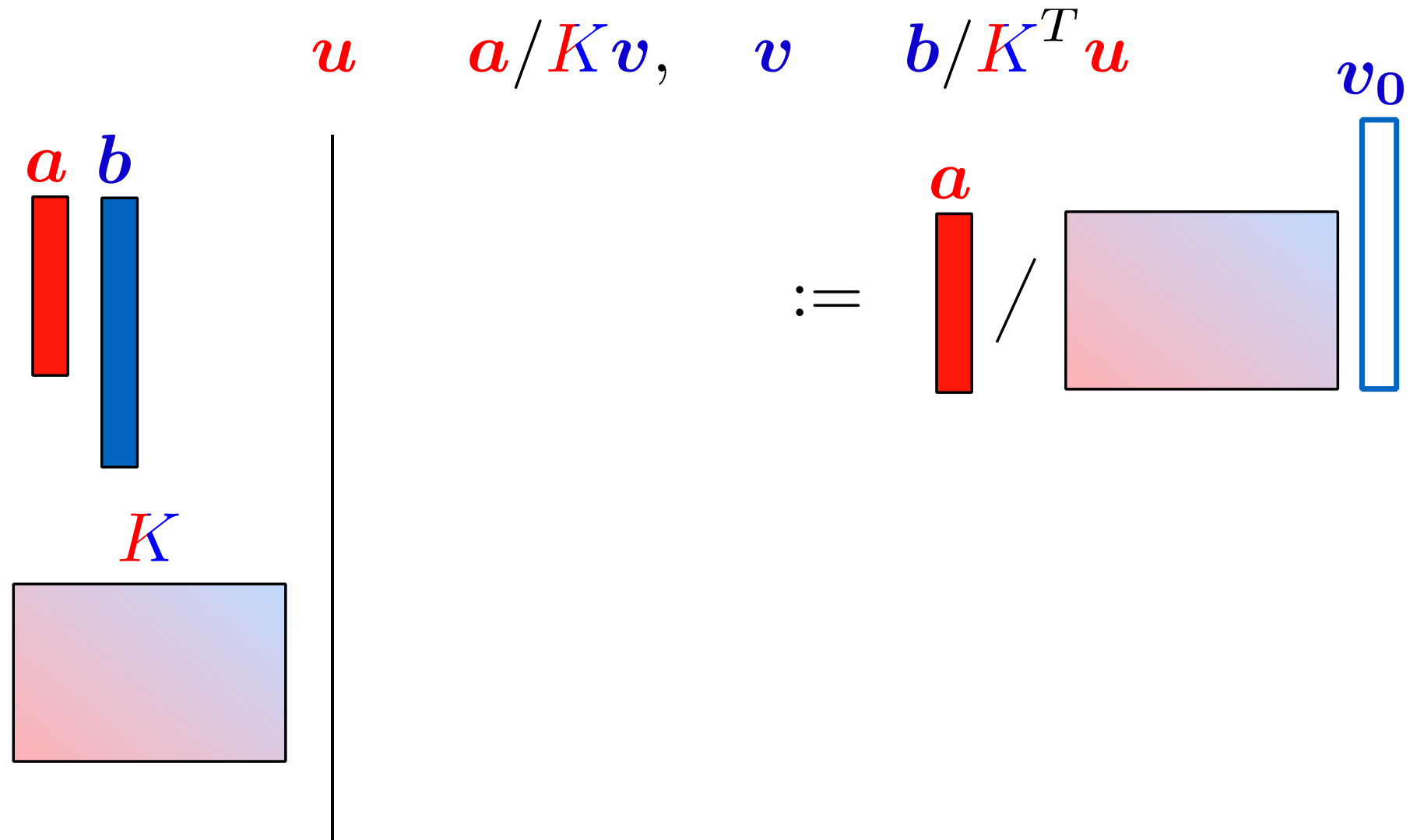
Repeat ① and ②

$O(N \log N)$

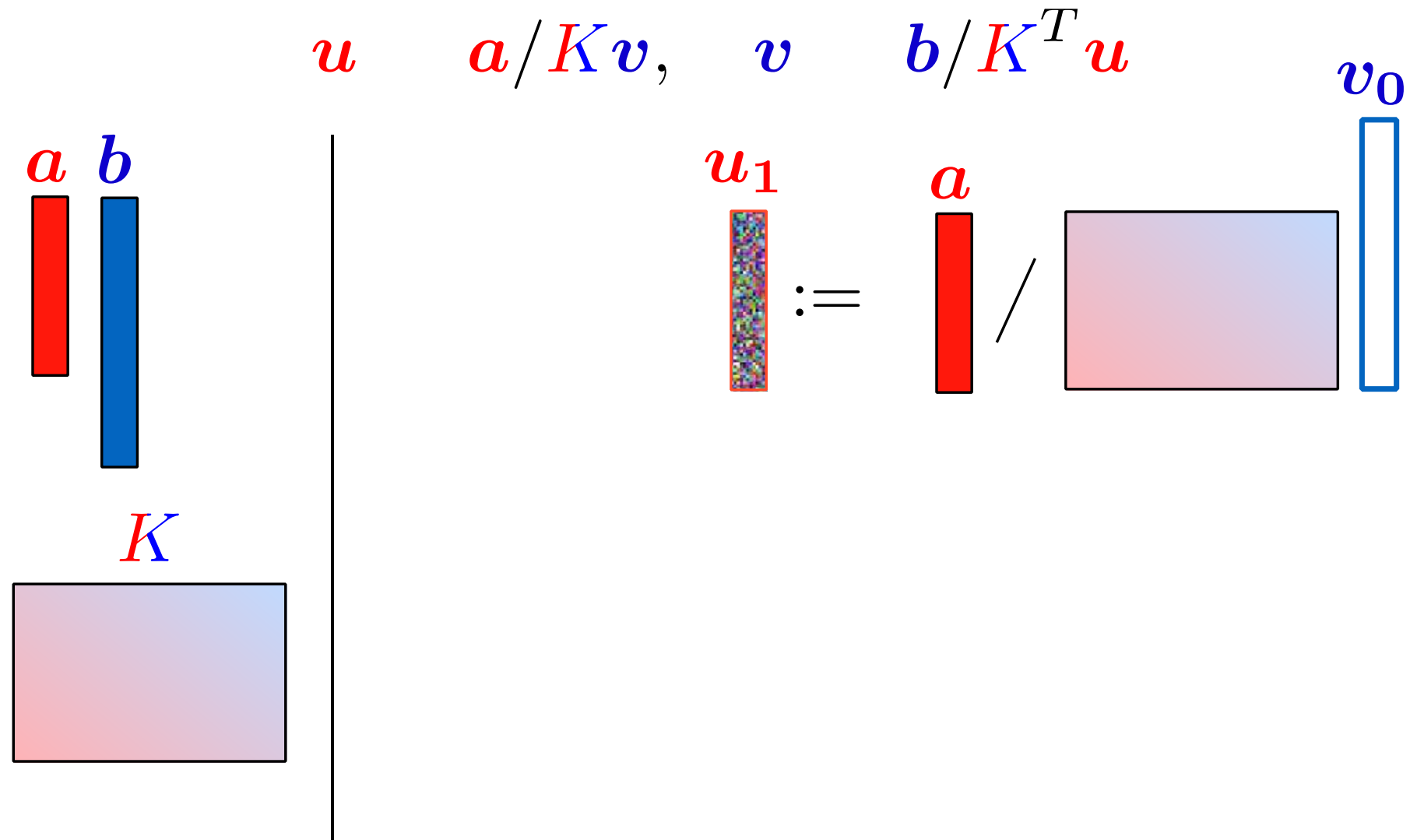
Fast & Scalable Algorithm



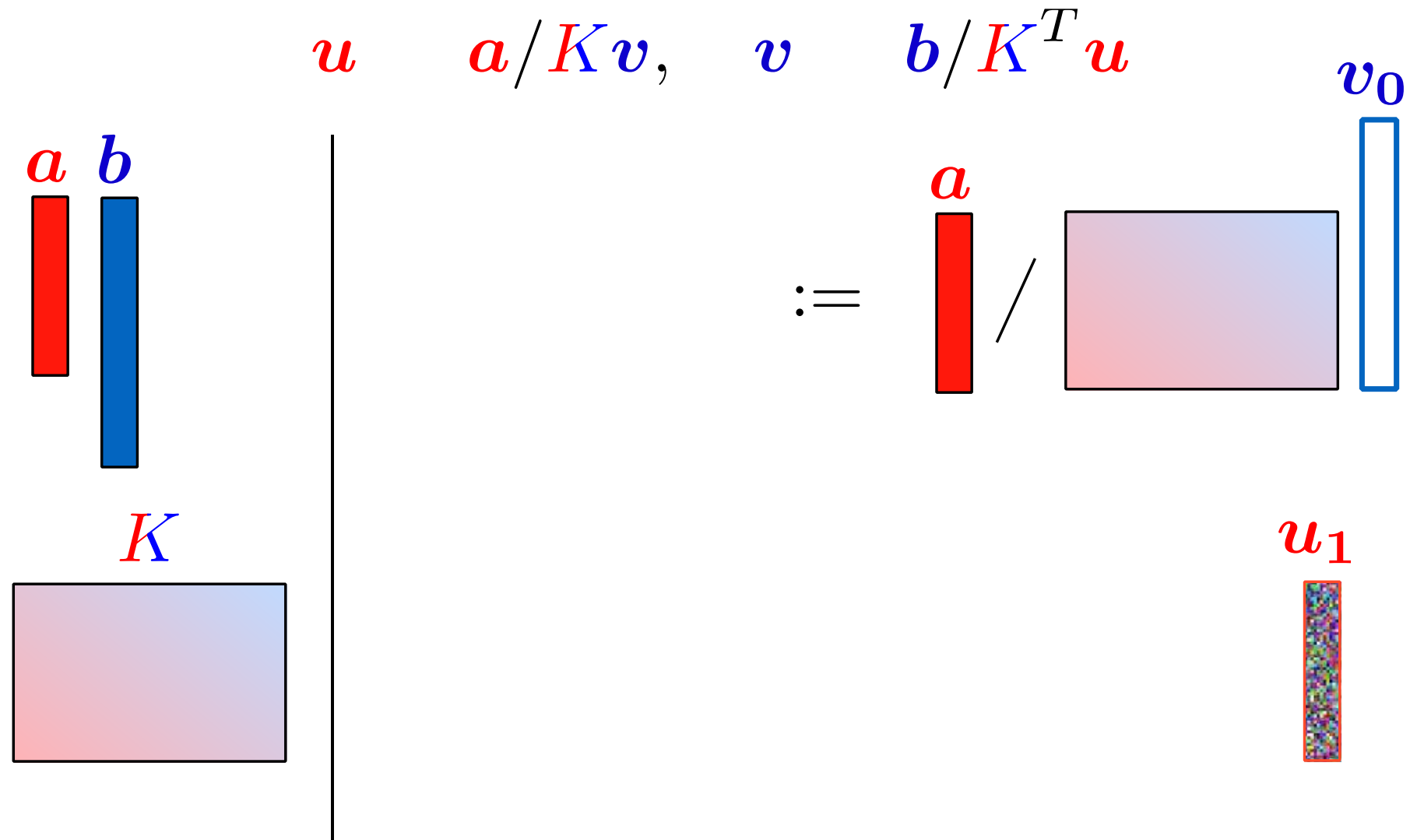
Fast & Scalable Algorithm



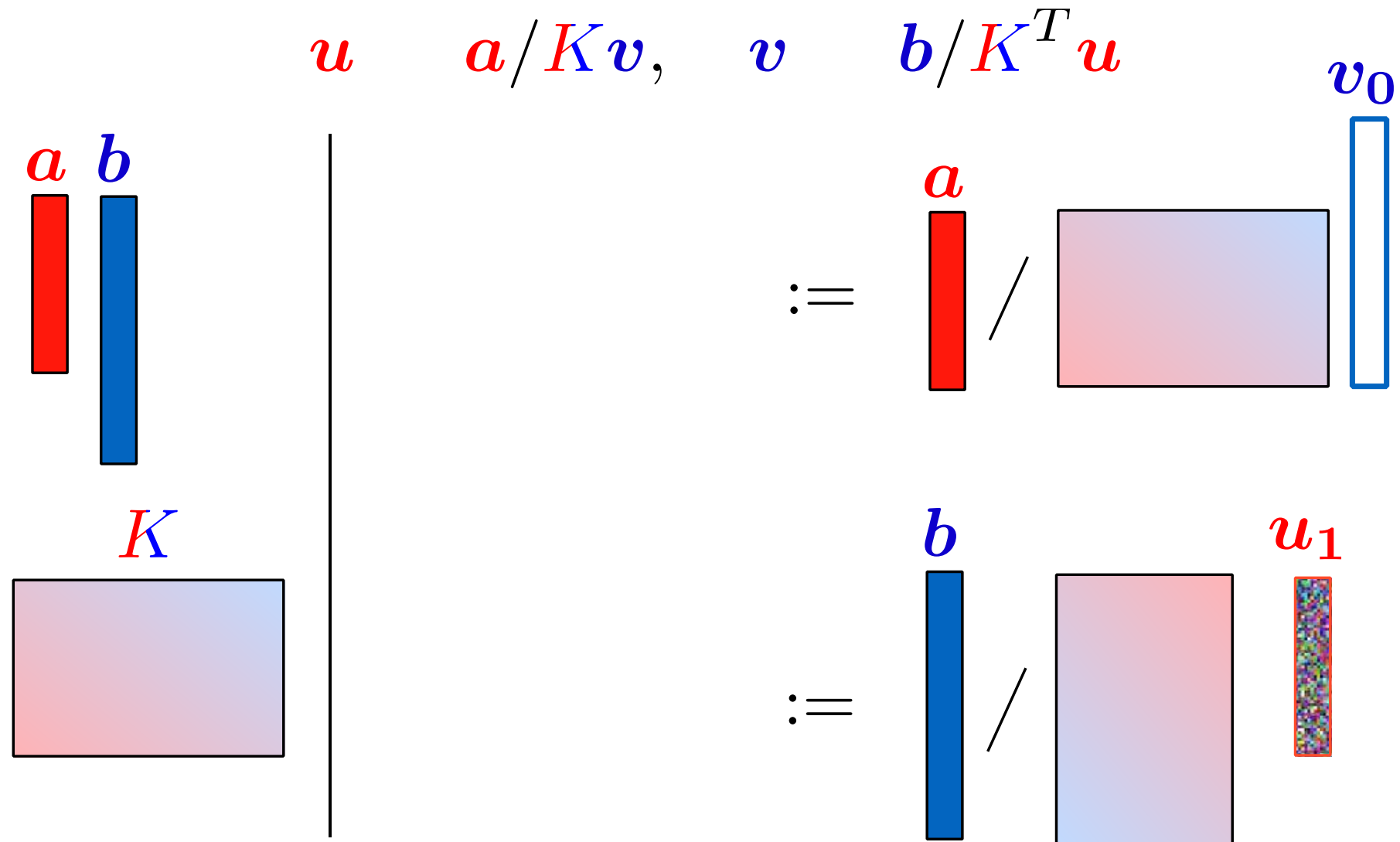
Fast & Scalable Algorithm



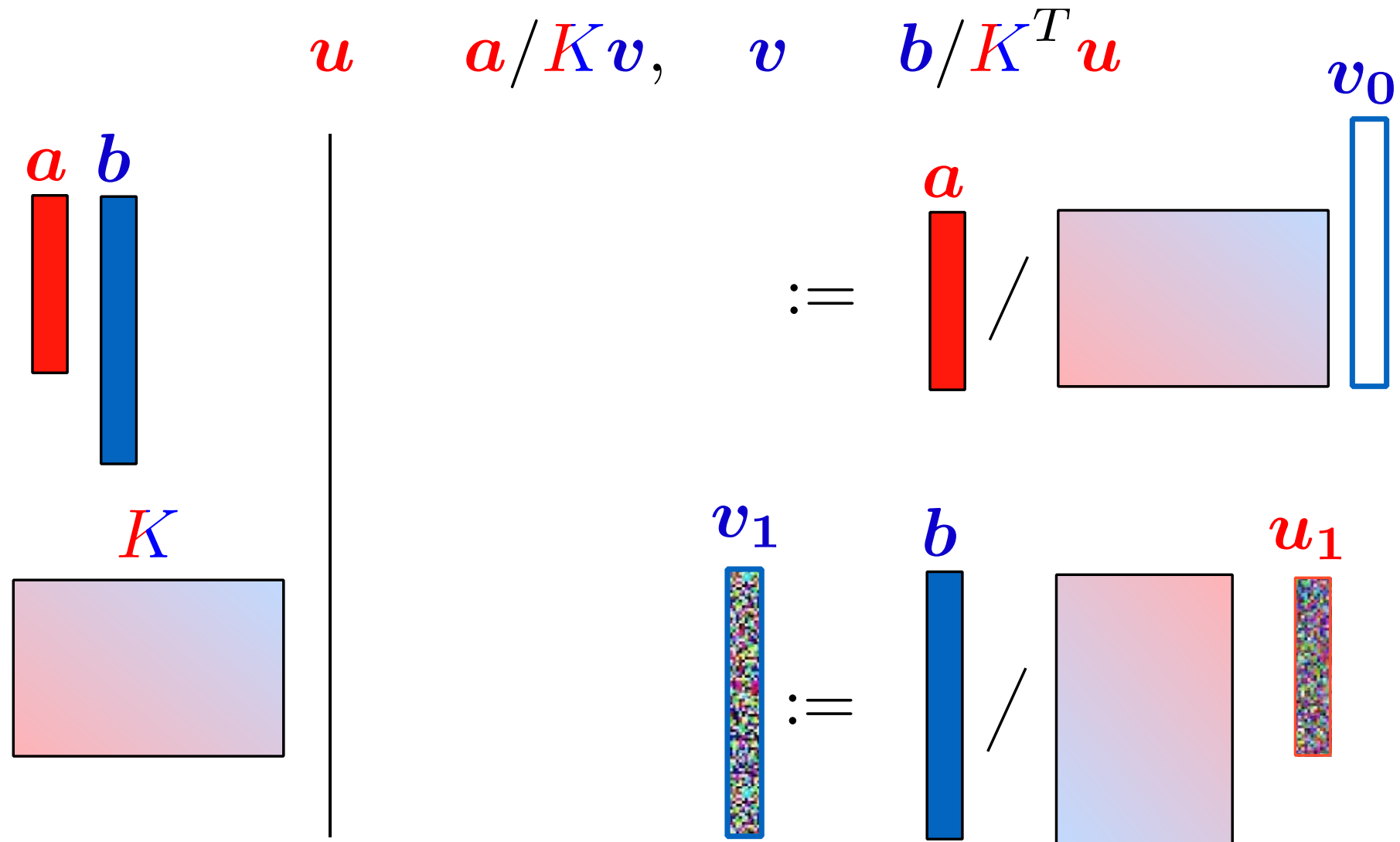
Fast & Scalable Algorithm



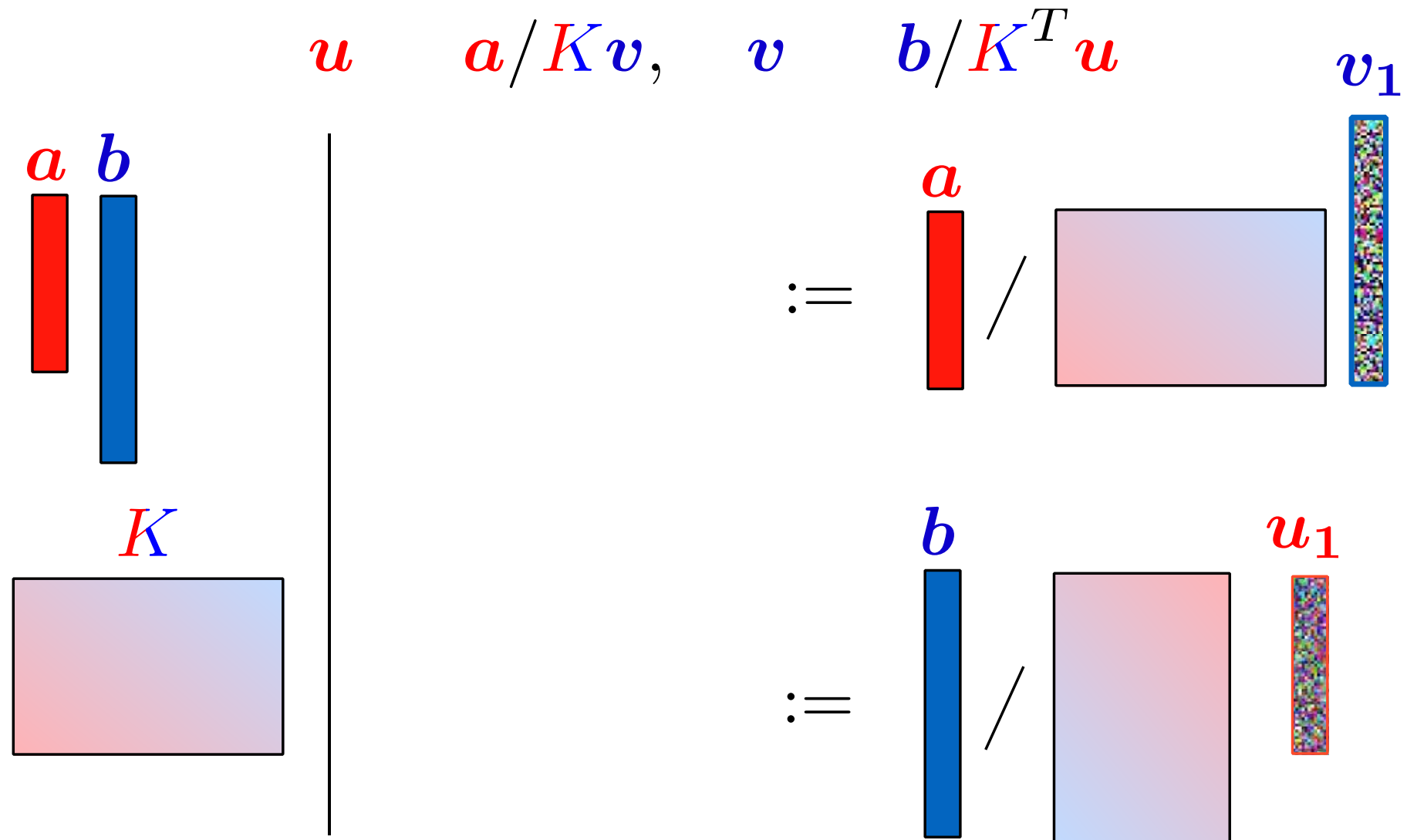
Fast & Scalable Algorithm



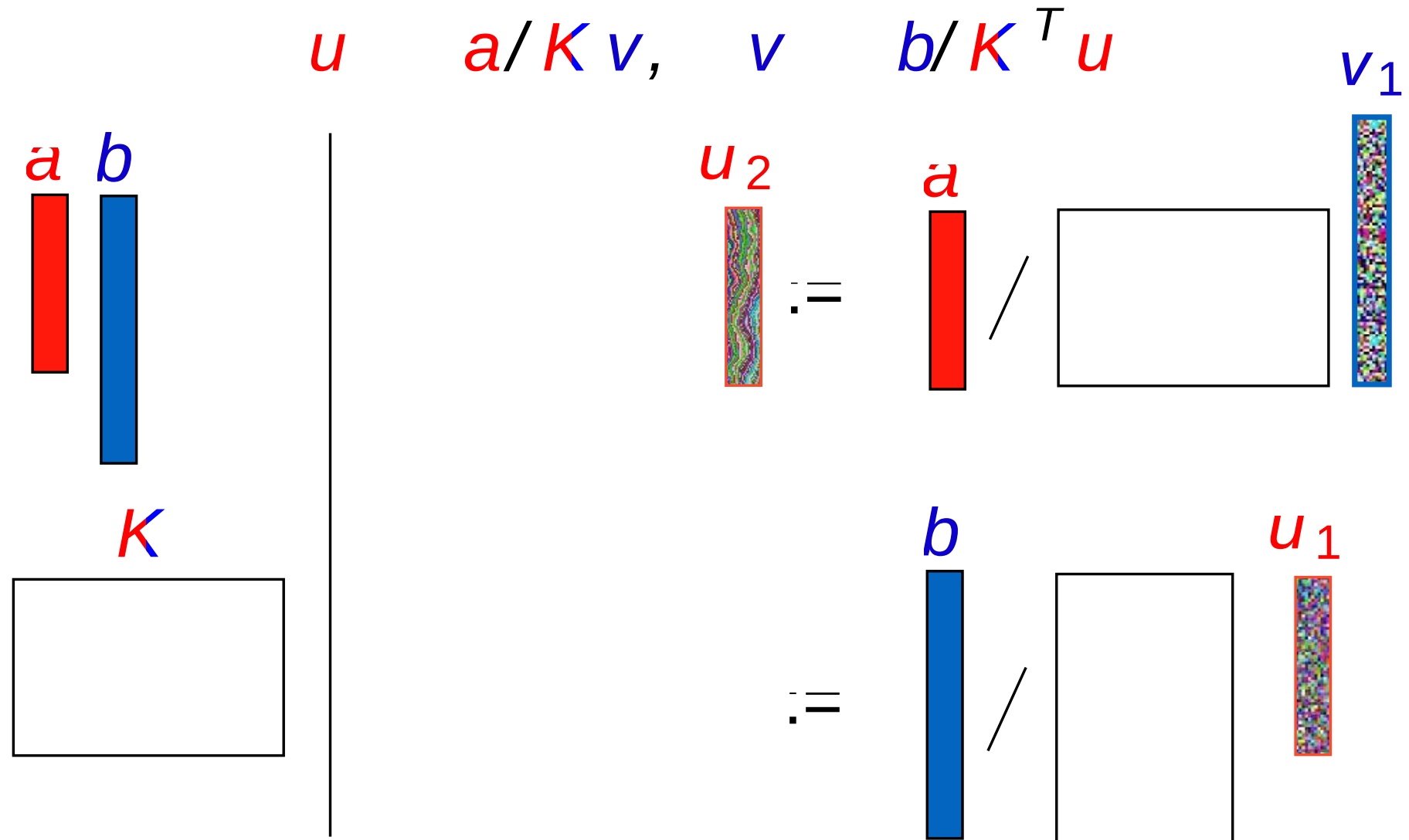
Fast & Scalable Algorithm



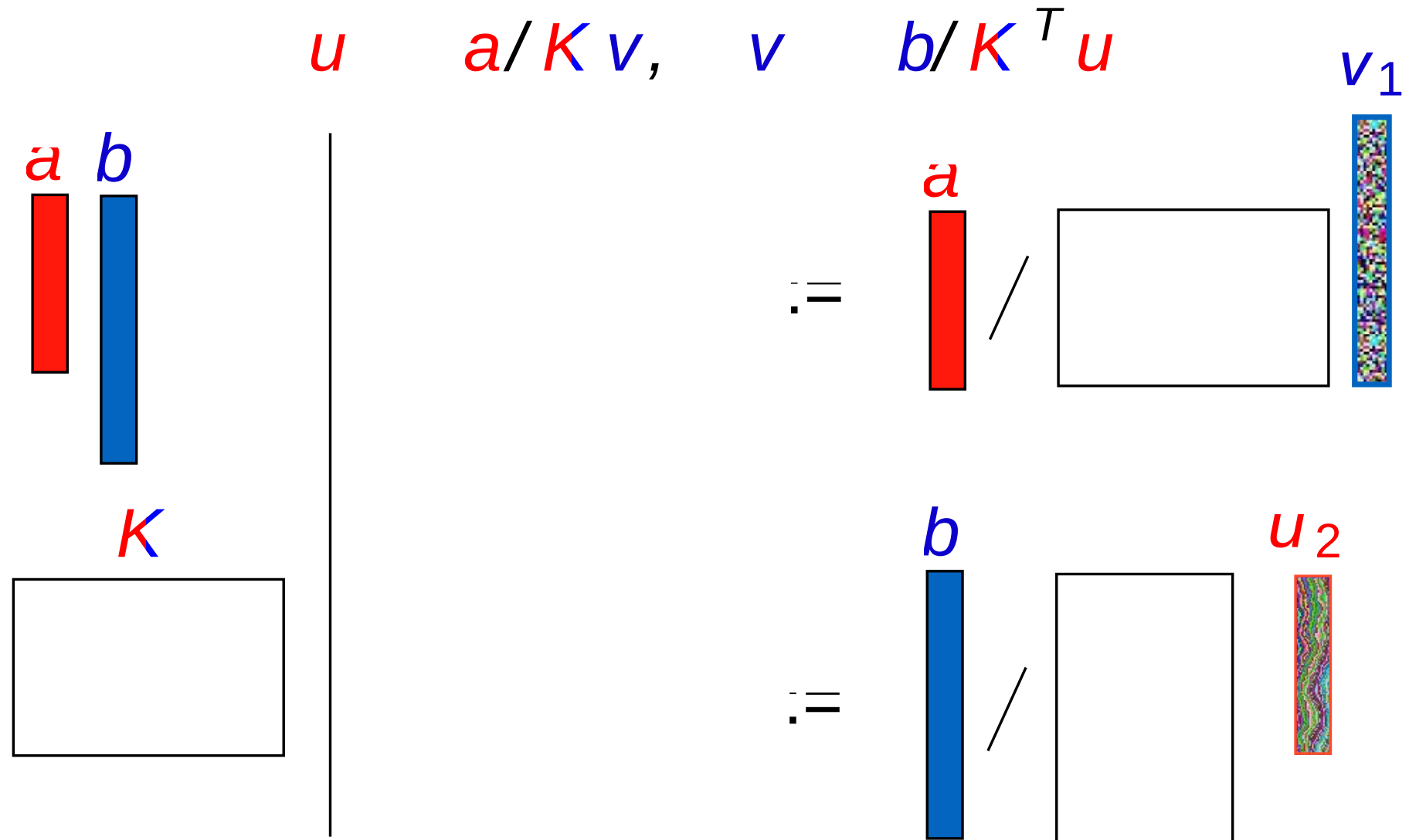
Fast & Scalable Algorithm



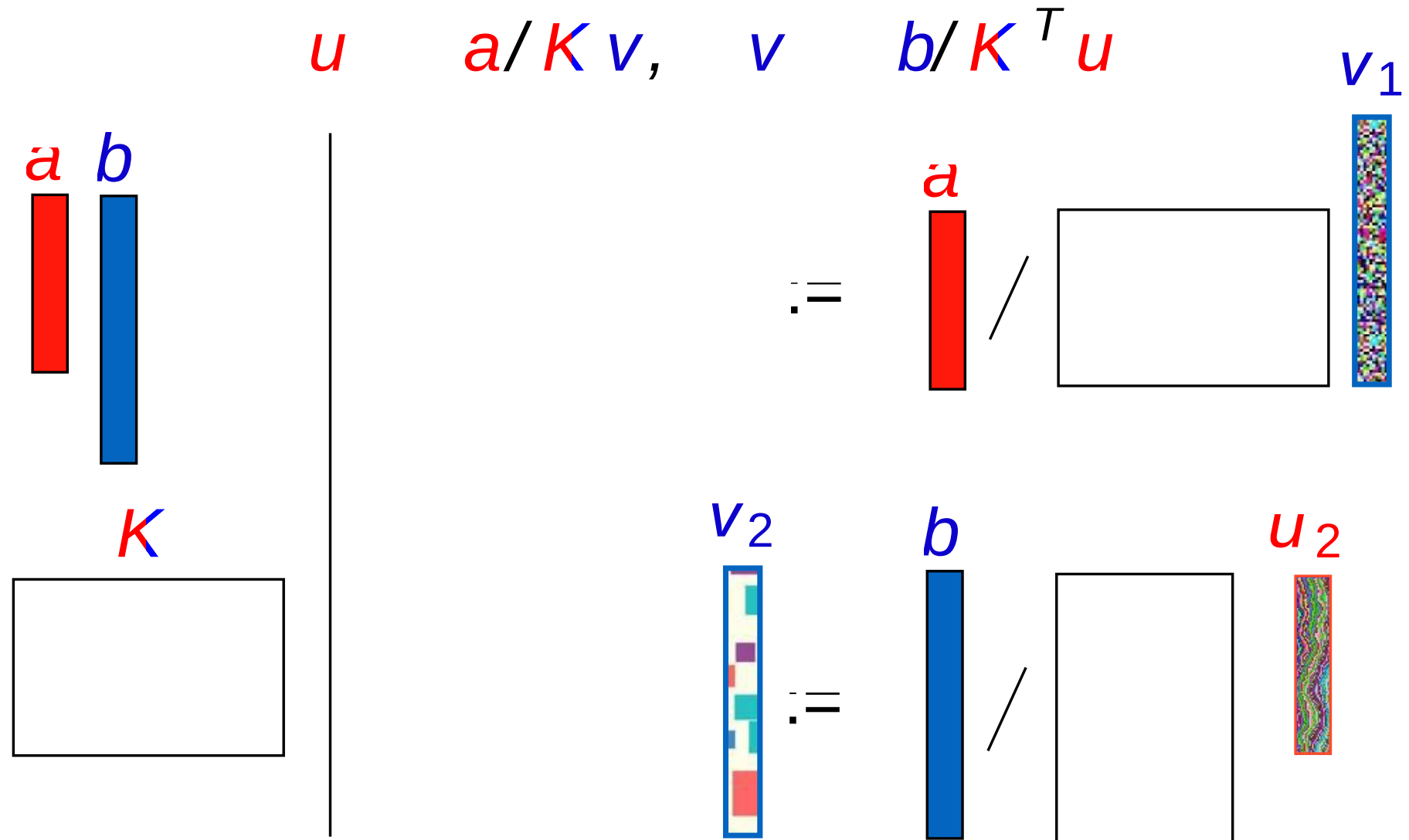
Fast & Scalable Algorithm



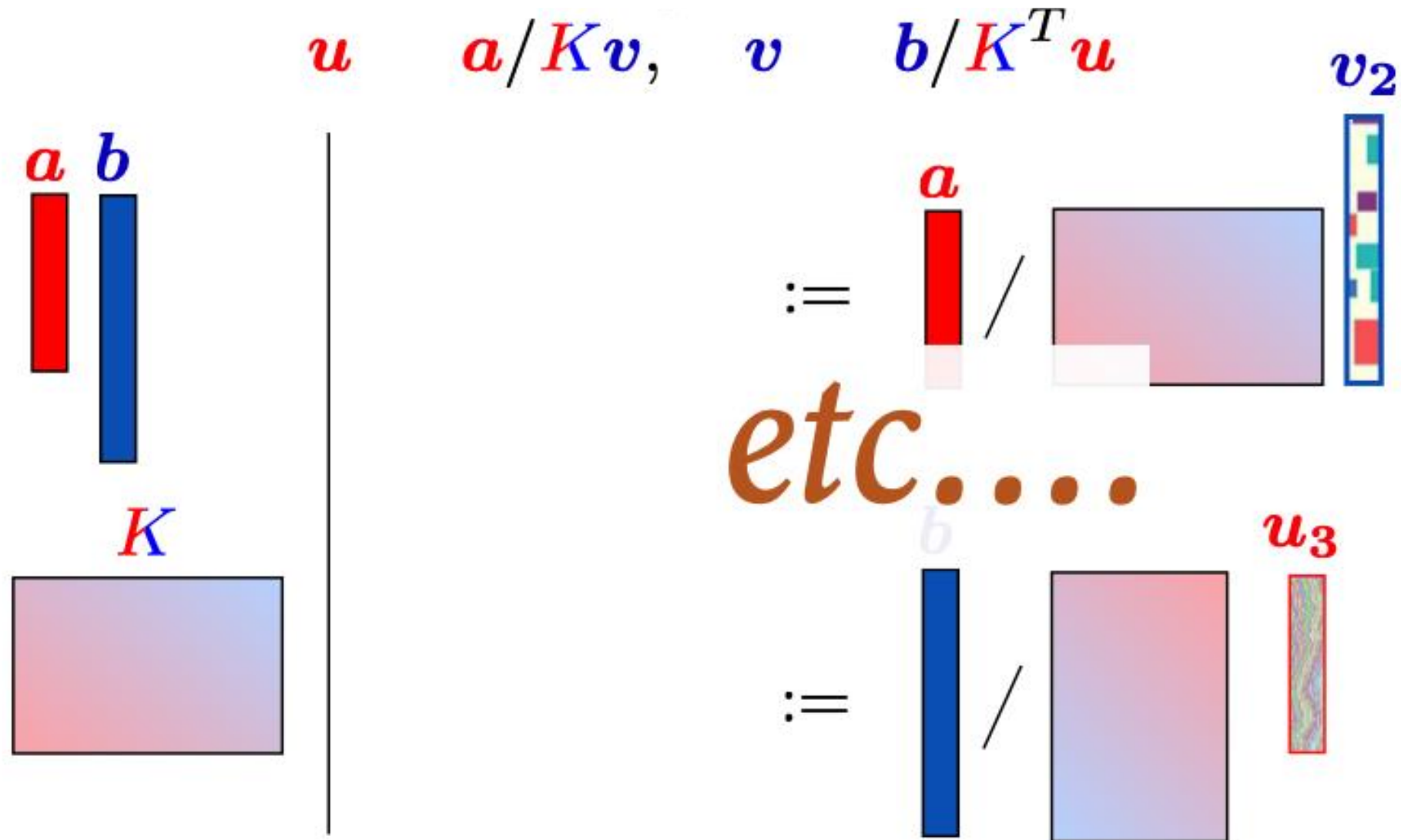
Fast & Scalable Algorithm

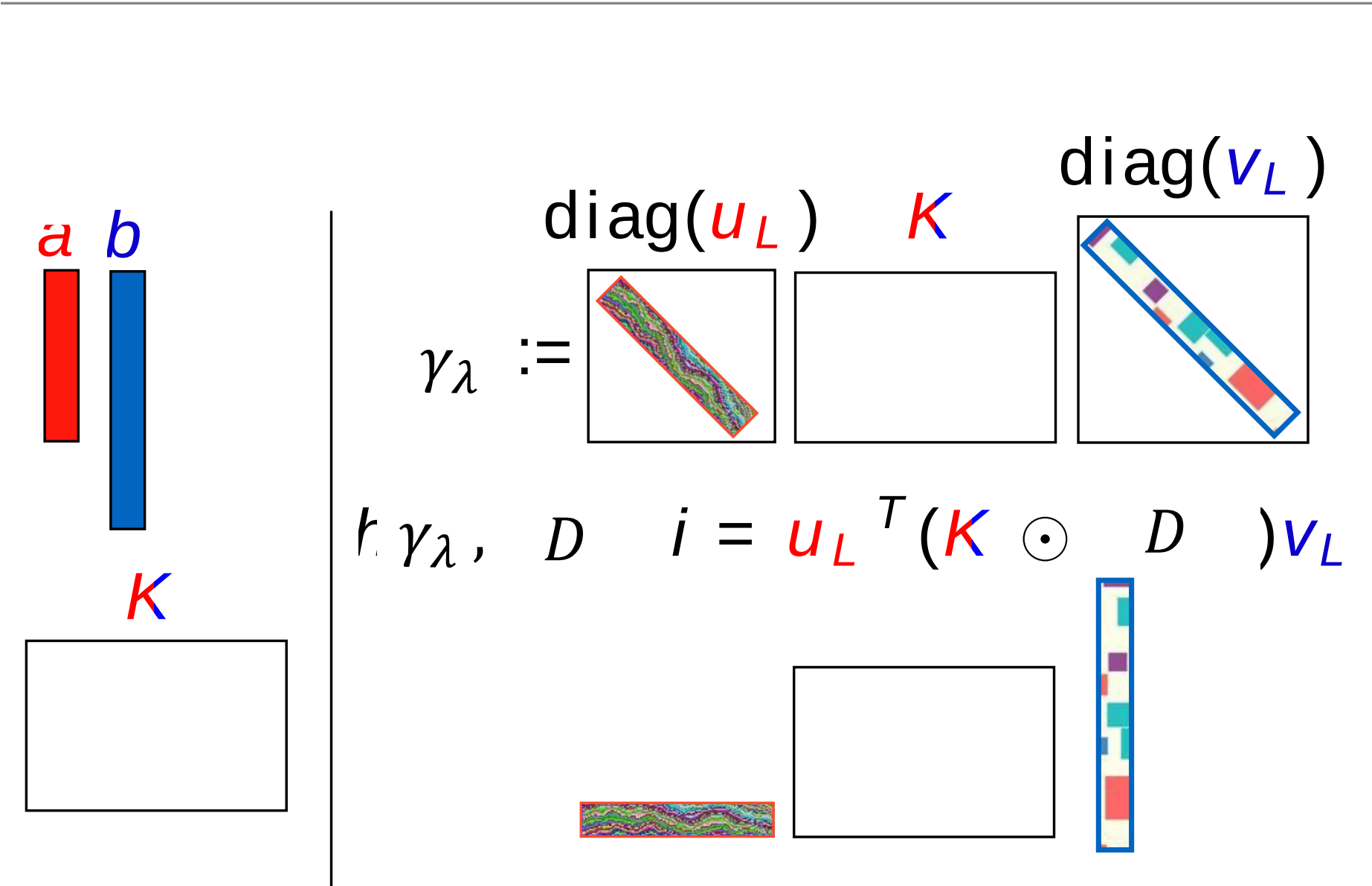


Fast & Scalable Algorithm



Fast & Scalable Algorithm





Solving the regularized problem **in the dual**

$$\textcircled{1} \quad \mathcal{L}(\gamma, \alpha, \beta) = \sum_{ij} \gamma_{ij} D_{ij} + \lambda \gamma_{ij} (\log \gamma_{ij} - 1) - \alpha^T (\gamma \mathbf{1}_m - a) - \beta^T (\gamma^T \mathbf{1}_n = b)$$

$$\textcircled{2} \quad \gamma_{ij} = \exp\left(\frac{\alpha_i}{\lambda}\right) K_{ij} \exp\left(\frac{\beta_j}{\lambda}\right) = u_i K_{ij} v_j$$

Injecting $\textcircled{2}$ in $\textcircled{1}$ and maximizing, we get:

$$\max_{\alpha, \beta} \sum_{ij} e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} D_{ij} + \lambda e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} \left(\frac{\alpha_i}{\lambda} - \frac{D_{ij}}{\lambda} + \frac{\beta_j}{\lambda} - 1 \right) - \alpha_i \left(e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} - a_i \right) - \beta_j \left(e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} - b_j \right)$$

$$= \max_{\alpha, \beta} \sum_{ij} \cancel{e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} D_{ij}} + \cancel{\alpha_i e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}}} - \cancel{e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} D_{ij}} + \cancel{\beta_j e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}}} - \lambda e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}} \\ - \cancel{\alpha_i e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}}} + \alpha^T a + \beta^T b - \cancel{\beta_j e^{\frac{\alpha_i}{\lambda}} K_{ij} e^{\frac{\beta_j}{\lambda}}}$$

$$= \max_{\alpha, \beta} \alpha^T a + \beta^T b - \lambda \left(e^{\frac{\alpha}{\lambda}} \right)^T K e^{\frac{\beta}{\lambda}}$$

Sinkhorn as a dual Algorithm!

From the primal, we know that: $\gamma_{ij} = \exp\left(\frac{\alpha_i}{\lambda}\right) K_{ij} \exp\left(\frac{\beta_j}{\lambda}\right) = u_i K_{ij} v_j$

Sinkhorn iterations

$$\begin{aligned} u &= a / K v \\ v &= b / K^T u \end{aligned}$$

Solving the dual: $\max_{\alpha, \beta} \alpha^T a + \beta^T b - \lambda \left(e^{\frac{\alpha}{\lambda}} \right)^T K e^{\frac{\beta}{\lambda}}$

$$\frac{\partial \alpha^T a + \beta^T b - \lambda \left(e^{\frac{\alpha}{\lambda}} \right)^T K e^{\frac{\beta}{\lambda}}}{\partial \alpha} = 0 \Leftrightarrow a - e^{\frac{\alpha}{\lambda}} K e^{\frac{\beta}{\lambda}} = 0 \Leftrightarrow e^{\frac{\alpha}{\lambda}} = \frac{a}{K e^{\frac{\beta}{\lambda}}} \Leftrightarrow u = a / K v$$

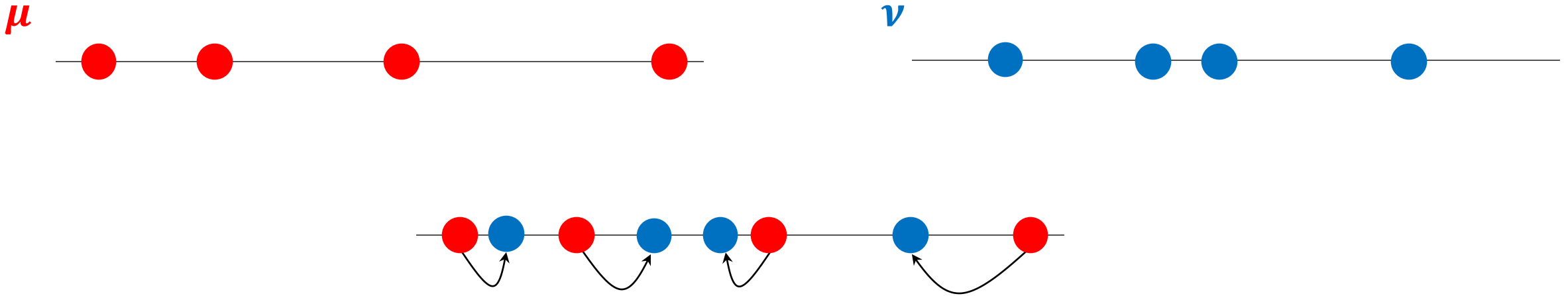
$$\frac{\partial \alpha^T a + \beta^T b - \lambda \left(e^{\frac{\alpha}{\lambda}} \right)^T K e^{\frac{\beta}{\lambda}}}{\partial \beta} = 0 \Leftrightarrow b - e^{\frac{\alpha}{\lambda}} K e^{\frac{\beta}{\lambda}} = 0 \Leftrightarrow e^{\frac{\beta}{\lambda}} = \frac{b}{K e^{\frac{\alpha}{\lambda}}} \Leftrightarrow v = b / K^T u$$

Sinkhorn = Block coordinate ascent on Dual

Wasserstein distance for 1D distributions

Discrete case

When $c(x, y)$ is a strictly convex and increasing function of $|x - y|$, the solution is given by the monotone rearrangement of μ and ν .



Very simple algorithm in $O(N \log N)$ by sorting points of μ and ν and summing the absolute values of differences.

Wasserstein distance for 1D distributions

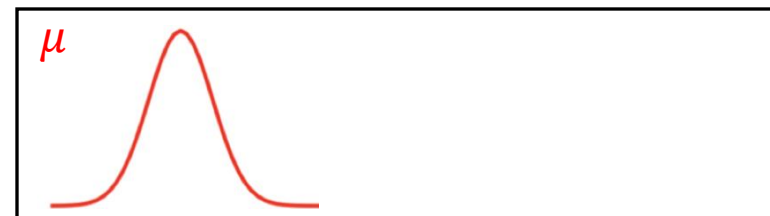
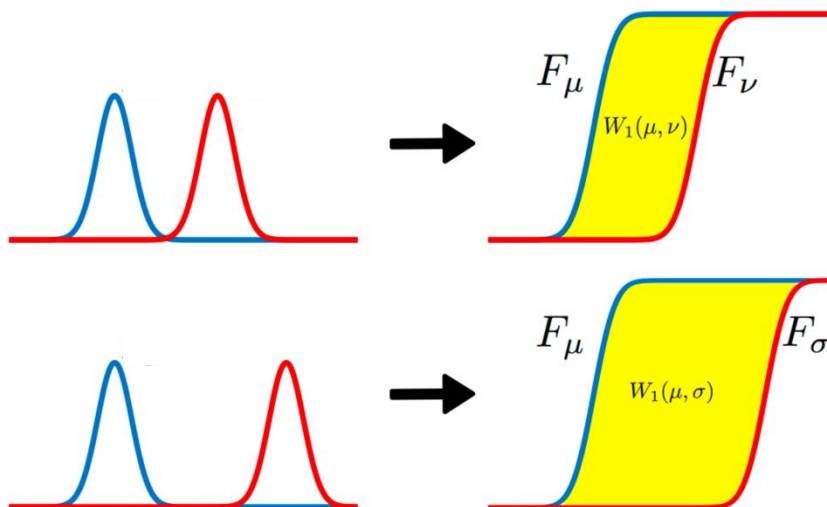
Continuous case

For a.c. 1D probability measures, the optimal transport map has a closed form solution.

Let F_μ and F_ν be the cumulative distribution functions,

$$\begin{cases} F_\mu(x) = \mu((-\infty, x)) \\ F_\nu(y) = \nu((-\infty, y)) \end{cases}$$

$$W_p^p(\mu, \nu) = \int_0^1 |F_\mu^{-1}(t) - F_\nu^{-1}(t)|^p dt$$



OUTLINE

- Introduction
- Monge Problem (1781)
- Kantorovich Problem - Wasserstein Distance
- Solving the OT problem (1942) – Regularized Wasserstein Distance
- **Gromov, Fused-Gromov and Diffusion Wasserstein Distances**
- Applications in Machine Learning and AI

Gromov-Wasserstein Distance

[Peyré et al. ICML 2016]

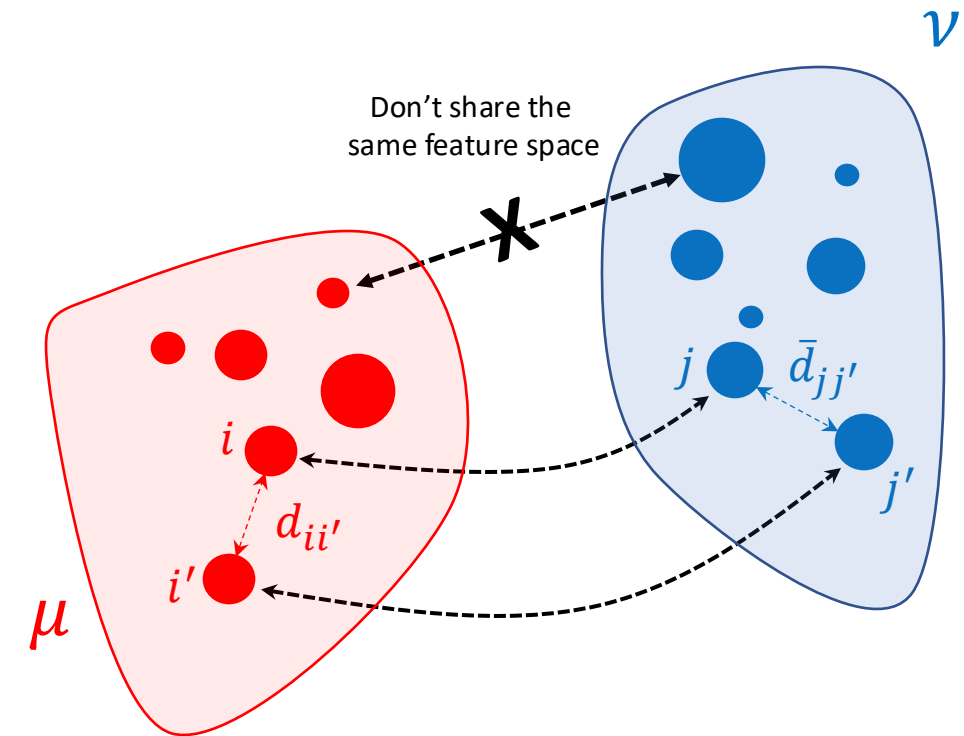
We assumed so far that both μ and ν lie in the same feature space.

The Gromov-Wasserstein distance allows to compare μ and ν even when we just have access to intra-distribution pairwise distances.

Def. *Gromov-Wasserstein*

$$GW^p(d, \mu, \bar{d}, \nu) = \min_{\gamma} \sum_{i, i', j, j'} |d_{ii'} - \bar{d}_{jj'}|^p \gamma_{ij} \gamma_{i'j'}$$

$$\text{s.t. } \gamma \in U_{ab} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = a; \gamma^T \mathbf{1}_n = b\}$$



Good news #1: Entropic regularization still possible!

Good news #2: Can be efficiently used for comparing graphs using, e.g. the shortest path between two vertices as intra-distance function. Note that **only the structure is taken into account.**

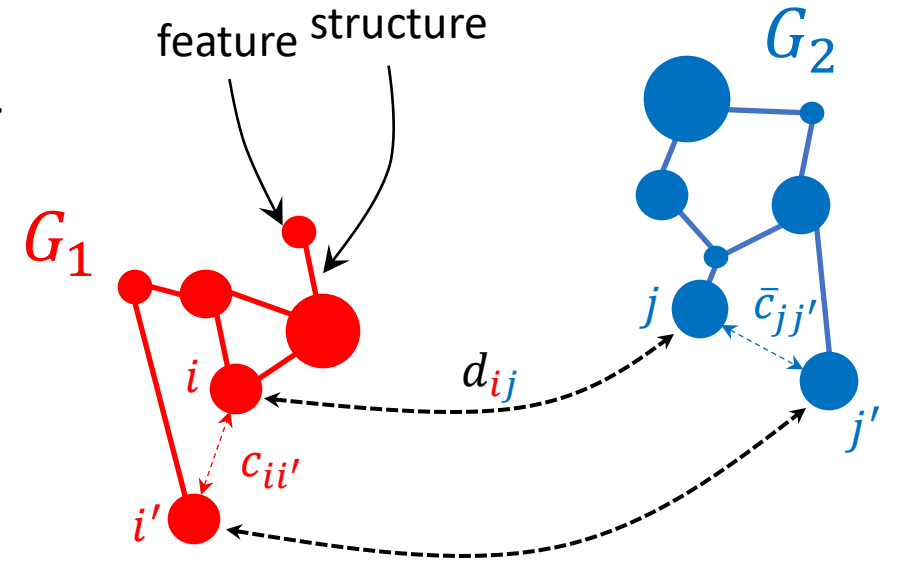
Fused-Gromov-Wasserstein Distance

[Vayer et al. ICML 2019]

Let suppose that μ and ν are respectively associated with two graphs G_1 and G_2 .

The Fused-Gromov-Wasserstein distance aims at benefiting from both the features **and** the structures of the graphs.

FGW acts as a generalization of the Wasserstein distance and the Gromov-Wasserstein distance

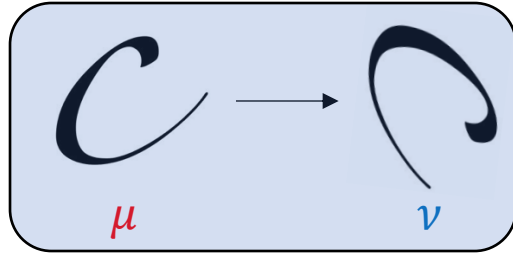


Def. *Fused-Gromov-Wasserstein*

$$FGW^p(d, \mathbf{c}, \mu, \bar{\mathbf{c}}, \nu) = \min_{\gamma} \sum_{i, i', j, j'} \overbrace{(1 - \alpha) d_{ij} \gamma_{ij}}^{\text{Wasserstein}} + \overbrace{\alpha |c_{ii'} - \bar{c}_{jj'}|^p \gamma_{ij} \gamma_{i'j'}}^{\text{Gromov}}$$

$$\text{s.t. } \gamma \in U_{ab} = \{\gamma \in \mathbb{R}_+^{n \times m}; \gamma \mathbf{1}_m = \mathbf{a}; \gamma^T \mathbf{1}_n = \mathbf{b}\}$$

Wasserstein versus Fused-Gromov Wasserstein



$$\min_B \sum_{i=1}^N k_i W_p^p(B, \nu_i)$$

Wasserstein
Barycenters

Wasserstein
Barycenters



Even though the entropic regularization can be used, FGW does not scale!

FGW
Barycenters

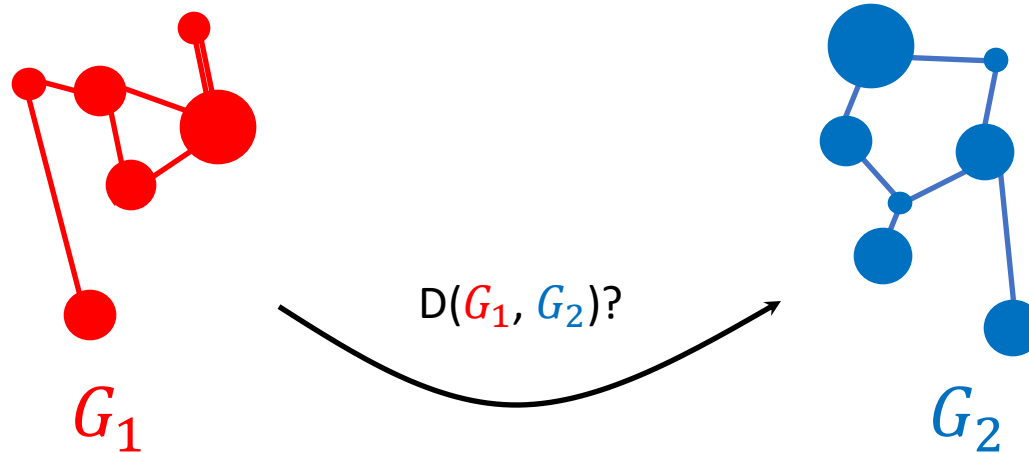


Can be very useful in chemical compounds or molecules modeling, brain connectivity, social networks, and many tasks in computer vision!!

Diffusion-Wasserstein Distance

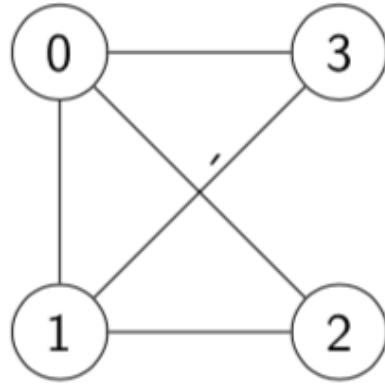
[Barbe et al. ECML 2020]

Goal: Define a new graph Wasserstein distance that scales well (**where γ is used only once**) and still captures both feature and structure information of attributed graphs



Our method is based on the spectral graph theory

Attributed Graph



$$V=\{0,1,2,3\}$$

$$E=\{(0,1), (0,2), \dots, (1,3)\}$$

Graph signal $\mathbf{x} : V \rightarrow \mathcal{X}$

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

Adjacency Matrix $\mathbf{A}$

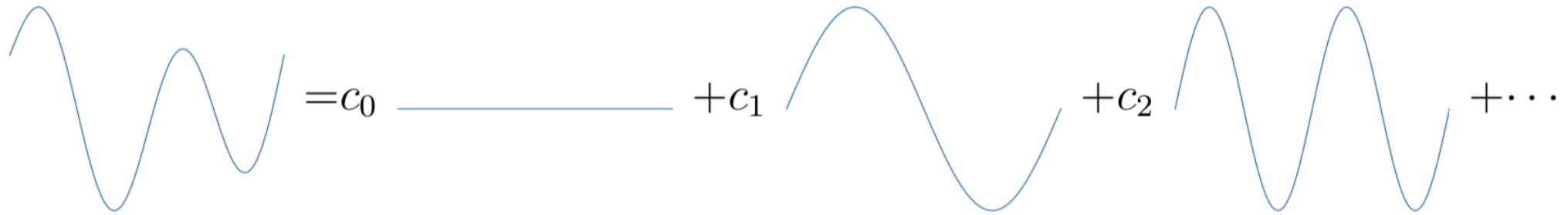
$$\begin{pmatrix} 3 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

Degree Matrix $\mathbf{D}$

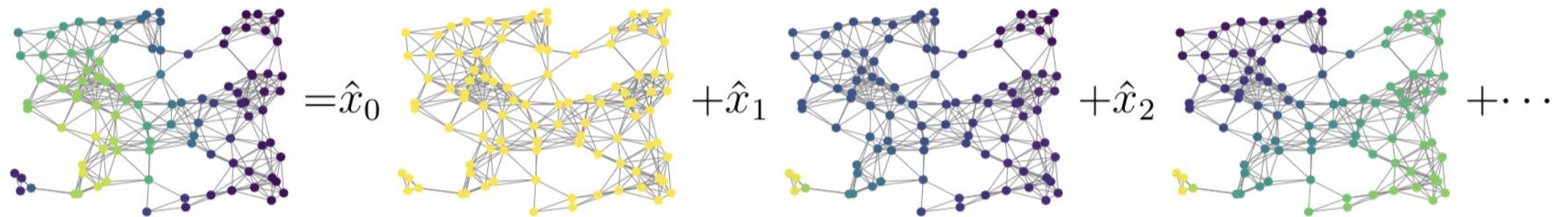
$$\begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & -1 \\ -1 & -1 & 2 & 0 \\ -1 & -1 & 0 & 2 \end{pmatrix}$$

Laplacian: $\mathbf{L} = \mathbf{D} - \mathbf{A}$

Spectral graph theory – Fourier decomposition



1d signal expressed as a linear combination in the Fourier basis



where the Laplacian of the graph is used to extend the notion of *Fourier basis, frequencies*.

Laplacian Diagonalization $L = P\Lambda P^T, \quad \Lambda = \text{diag}(\lambda_i)_{1 \leq i \leq n}$

The first eigenvector is constant, the second varies slowly, the next ones vary quicker and quicker.

The Laplacian plays a key role in the « heat diffusion » process in graphs

Heat Diffusion in Graphs

The heat diffusion of the signal X (defined on the nodes) in a graph is described by the following first order differential equation:

$$\forall i, \frac{dX_i}{dt} = \sum_{(i,j) \in E} (X_j - X_i) \quad (\text{rate of change})$$

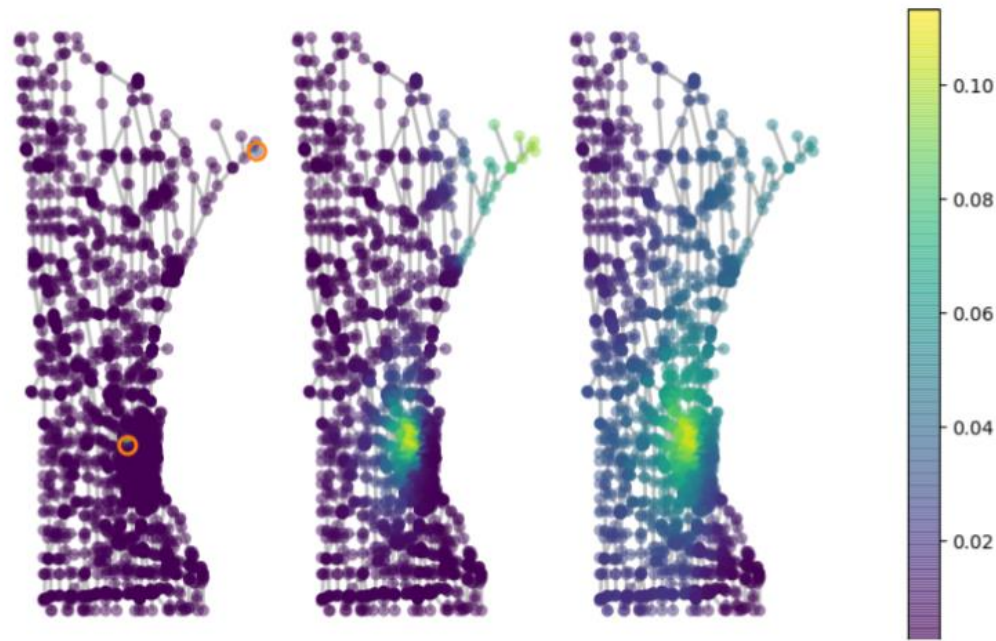
$$X(0) = v \quad (\text{initial condition})$$

The signal value of « cold » nodes (lower value than their neighbors) is increased and vice-versa.

It admits the following closed-form solution:

$$X(\tau) = \underbrace{\exp(-\tau L)}_{\substack{\text{Diffusion operator} \\ \text{or} \\ \text{Heat kernel}}} X(0)$$

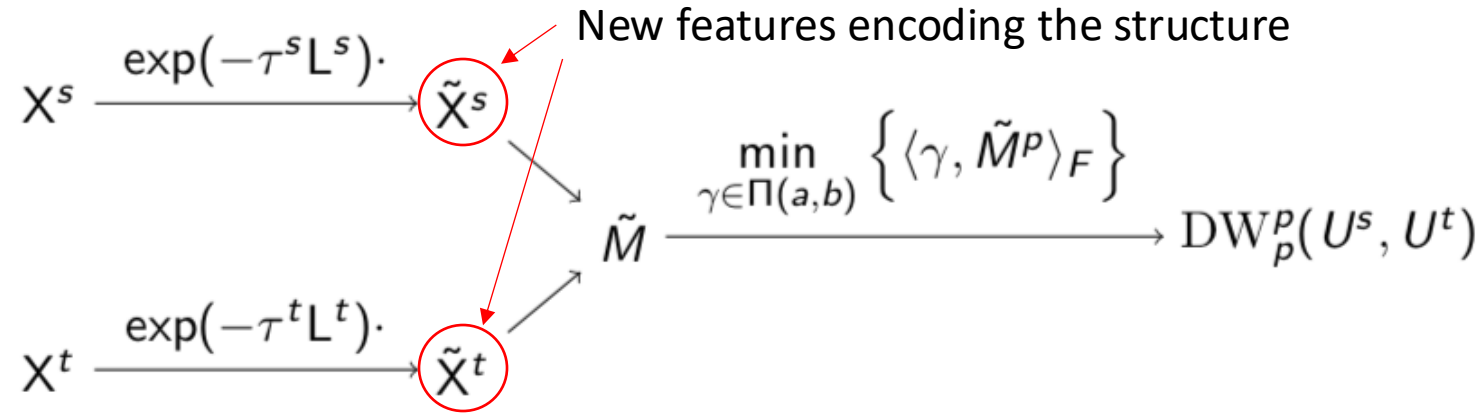
Diffusion time



The heat diffusion encodes the structure information of the graph!

Our idea: define a Wasserstein distance based on the heat diffusion applied on the graph signal

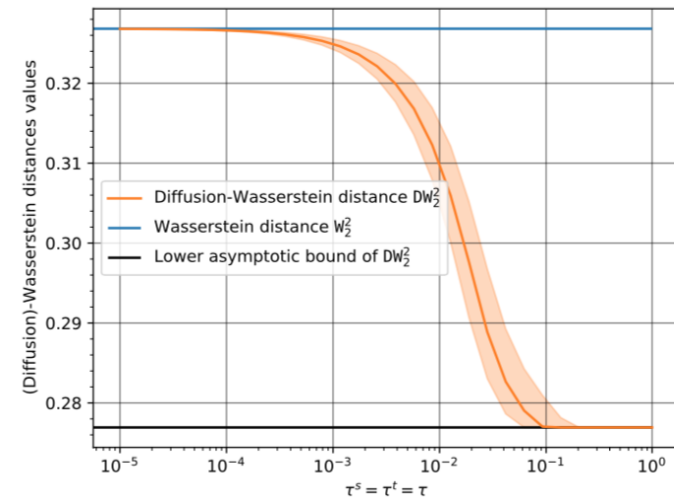
Diffusion-Wasserstein Distances for Attributed Graphs



$$DW_p^p(U^s, U^t) = \min_{\gamma \in \Pi(U^s, U^t)} \left\{ \sum_{i=1}^m \sum_{j=1}^n \tilde{M}_{ij}^p \gamma_{ij} \right\},$$

$$\text{avec } \tilde{M}_{ij} = \|(e^{-\tau^s L^s} X^s)_i - (e^{-\tau^t L^t} X^t)_j\|$$

- ▶ DW defines a pseudo-metric.
- ▶ $\lim_{\tau^s, \tau^t \rightarrow 0} DW(U^s, U^t) = W(U^s, U^t)$
- ▶ $\lim_{\tau^s, \tau^t \rightarrow \infty} DW(U^s, U^t) = \|\frac{1}{m} \sum_{i=1}^m X_i^s - \frac{1}{n} \sum_{j=1}^n X_j^t\|$



OUTLINE

- Introduction
- Monge Problem (1781)
- Kantorovich Problem (1942) - Wasserstein Distance
- Solving the OT problem – Regularized Wasserstein Distance
- Gromov, Fused-Gromov and Diffusion Wasserstein Distances
- **Applications in Machine Learning and AI**

Applications of Optimal Transport

Computer-Vision - Generative Models

Examples of images generated by a **Wasserstein GAN** [Arjovsky et al 2017]



Jiqing Wu et al. 2018 – ECCV 2018

φ is 1-Lipschitz continuous if:

$$\forall x, y \\ |\varphi(x) - \varphi(y)| \leq |x - y|$$

$$\min_G \sup_{\varphi \in LIP_1} \mathbb{E}_{x \sim \mu} [\varphi(x)] - \mathbb{E}_{z \sim \mathcal{N}(0, I)} [\varphi(G(z))]$$

= dual problem of OT (cf Kantorovich-Rubinstein theorem)

φ can be learned as a neural network constrained to the set LIP_1

Applications of Optimal Transport

Computer-Vision - Generative Models

Recall that the original Kantorovich problem is a LP problem

Dual Optimal Problem (discrete case)

$$\max_{\substack{\alpha \in \mathbb{R}^n, \beta \in \mathbb{R}^m \\ \alpha_i + \beta_j \leq D_{ij}}} \alpha^T a + \beta^T b$$

Dual Optimal Problem (continuous case)

$$\max_{\substack{\varphi, \psi \\ \varphi(x) + \psi(y) \leq D(x, y)}} \int \varphi d\mu + \int \psi d\nu$$

Kantorovich-Rubinstein theorem

When the cost function is the ℓ_1 norm, $D(x, y) = |x - y|$, the dual problem can be simplified into:

$$\max_{\varphi \in LIP_1} \int \varphi d\mu - \int \varphi d\nu = \max_{\varphi \in LIP_1} \mathbb{E}_{x \sim \mu} [\varphi(x)] - \mathbb{E}_{y \sim \nu} [\varphi(y)]$$

where LIP_1 is the set of 1-Lipschitz functions

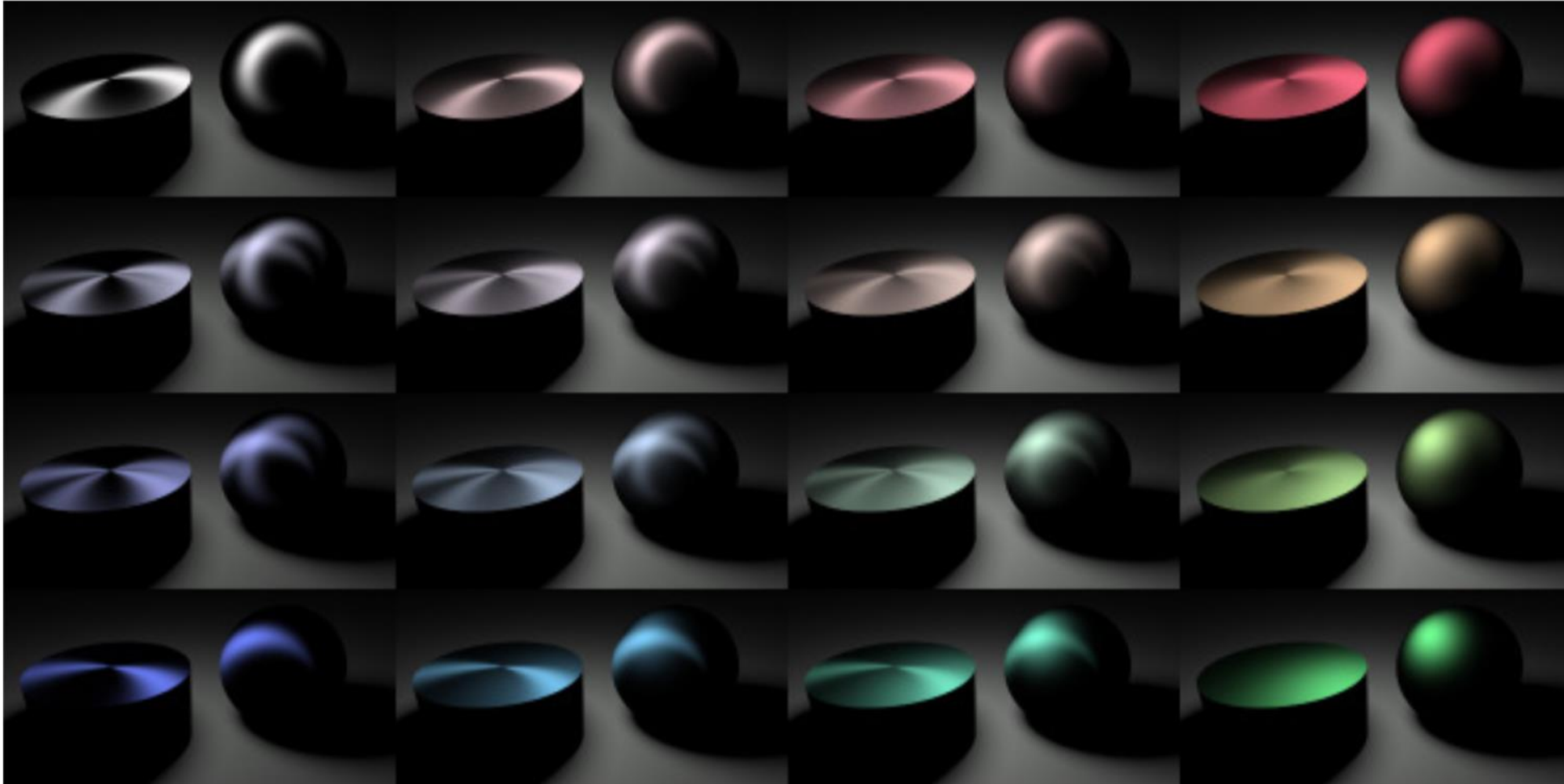


φ can be learned as a neural network constrained to the set LIP_1 $\longrightarrow$

Wasserstein GAN
[Arjovsky et al 2017]

Applications of Optimal Transport

Imaging

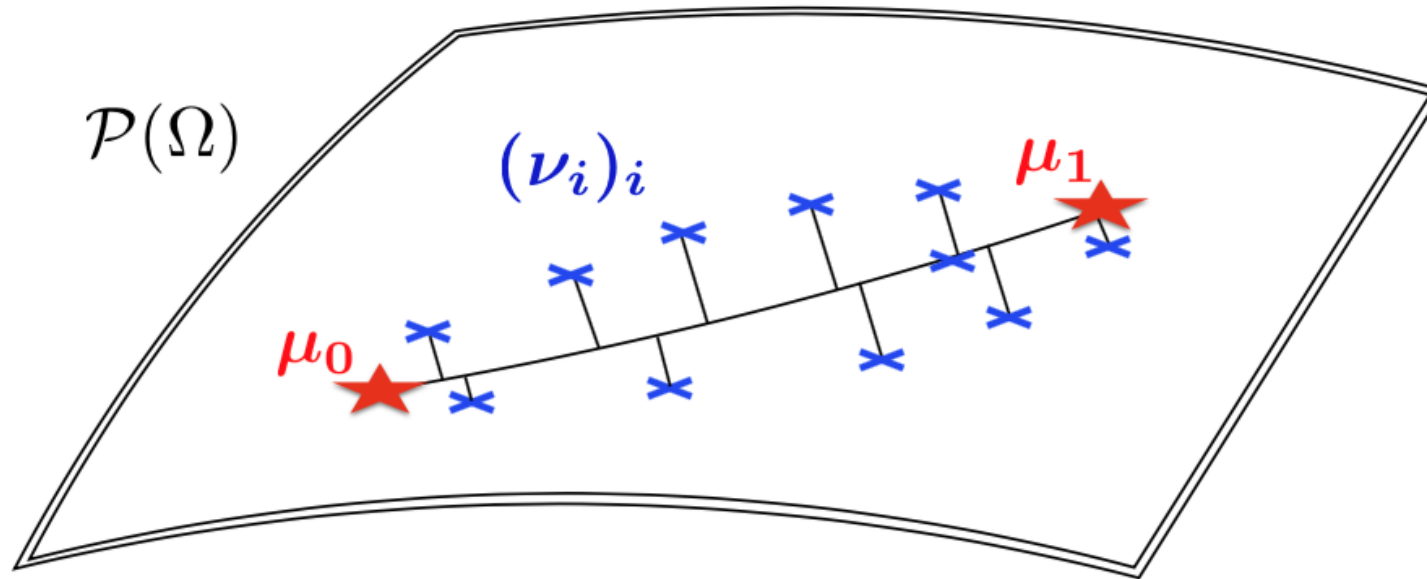


[Solomon'15]

Applications of Optimal Transport

Wasserstein PCA

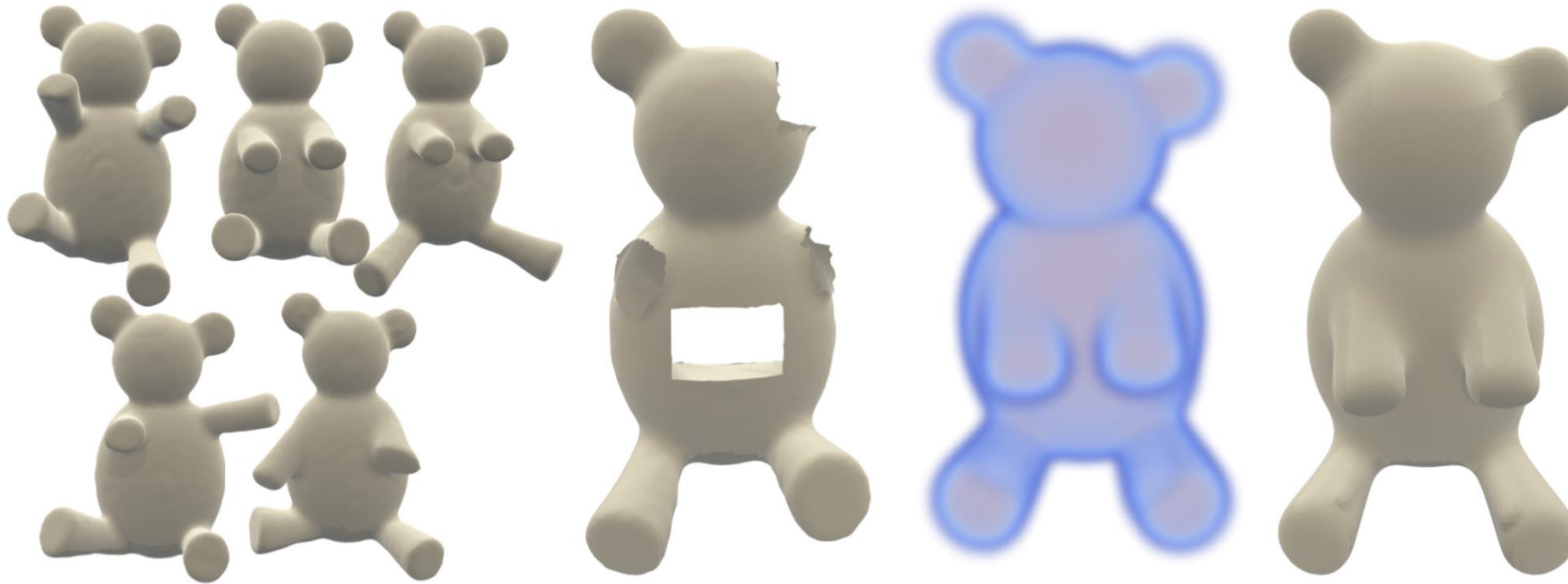
$$\min_{\mu_0, \mu_1} \sum_{i=1}^N \min_t W_2^2(\rho_{\mu_0 \rightarrow \mu_1}^t, \nu_i)$$



Cuturi-Solomon- Tutorial NIPS'17

Applications of Optimal Transport

Volume Reconstruction



Shape database
 $(p_1, \dots, p_5)$

Input shape q

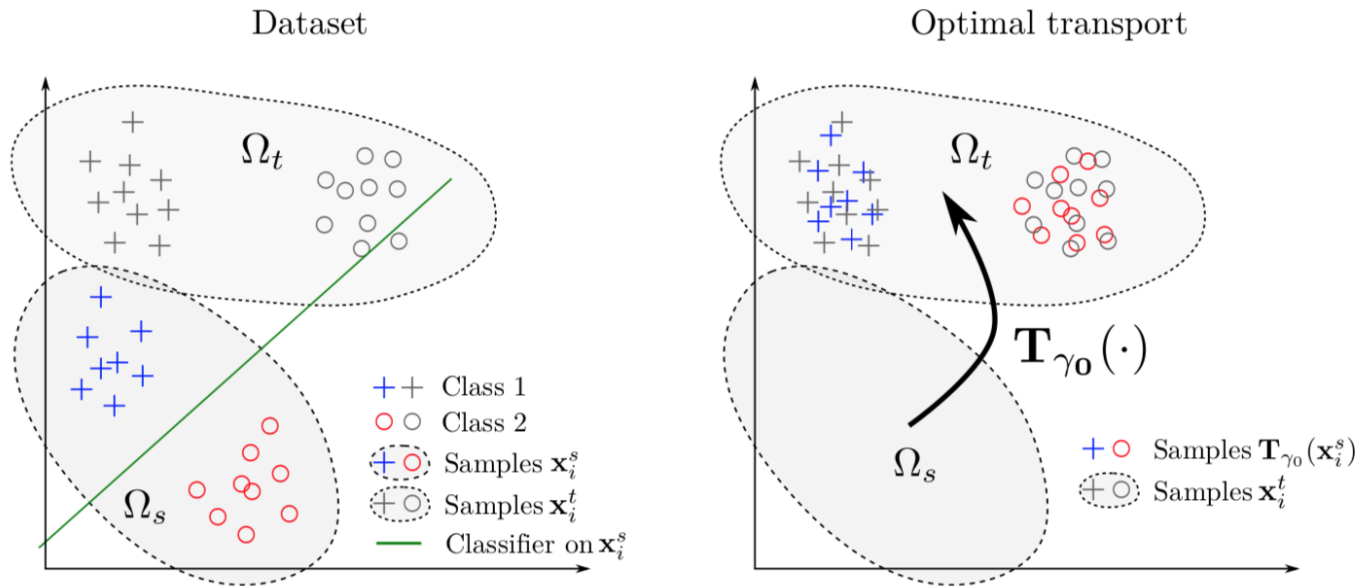
Projection
 $P(\lambda)$

Iso-surface

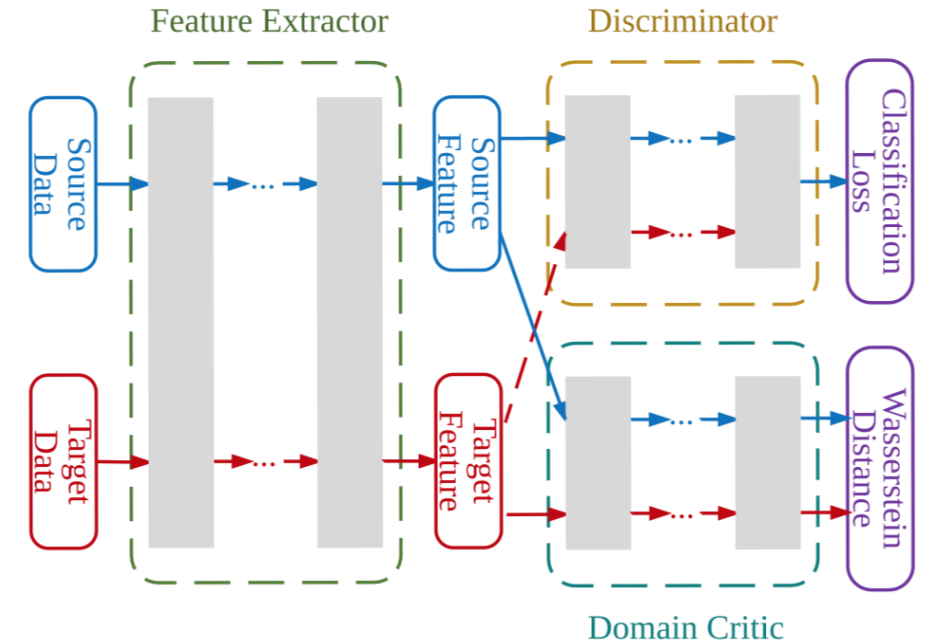
[Bonneel'16]

Applications of Optimal Transport

Domain Adaptation



Courty et al. 2016



Shen et al. 2018

One needs to prevent the transport of two source instances with different labels to the same target instance.

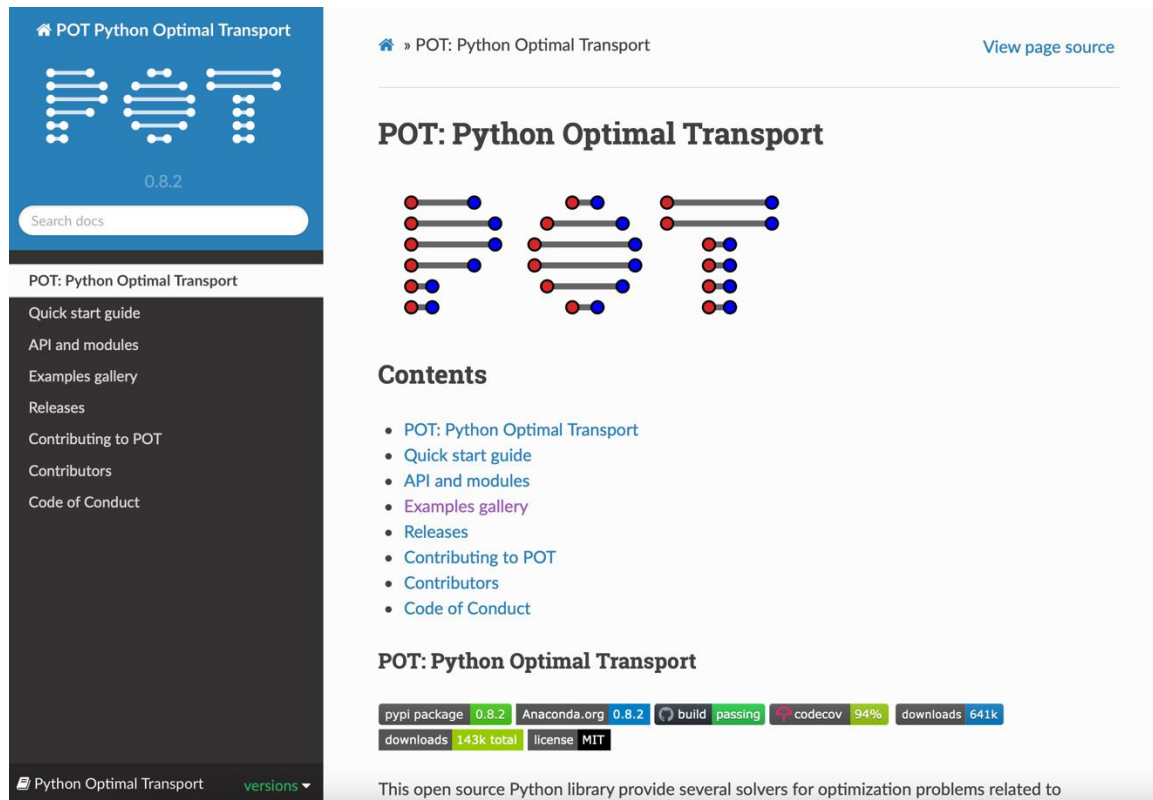
Tools for playing with Optimal Transport

POT: Python Optimal Transport

<https://pythonot.github.io> : open source Python library providing several solvers for optimization problems related to OT

OTT: Optimal Transport Tools

<https://github.com/google-research/ott> : JAX toolbox that bundles a few utilities to solve optimal transport problems.



The image shows a side-by-side comparison of the POT Python Optimal Transport documentation. On the left is a mobile view of the website, and on the right is a desktop view.

Mobile View (Left):

- Header: POT Python Optimal Transport
- Logo: A stylized logo consisting of three sets of horizontal lines with dots at the ends, representing optimal transport plans.
- Version: 0.8.2
- Search bar: Search docs
- Navigation menu (dark background):
 - POT: Python Optimal Transport
 - Quick start guide
 - API and modules
 - Examples gallery
 - Releases
 - Contributing to POT
 - Contributors
 - Code of Conduct
- Footer: Python Optimal Transport versions

Desktop View (Right):

- Header: POT: Python Optimal Transport
- Logo: The same stylized logo as the mobile view.
- Contents section:
 - POT: Python Optimal Transport
 - Quick start guide
 - API and modules
 - Examples gallery
 - Releases
 - Contributing to POT
 - Contributors
 - Code of Conduct
- Footer section:
 - pypi package 0.8.2
 - Anaconda.org 0.8.2
 - build passing
 - codecov 94%
 - downloads 641k
 - downloads 143k total
 - license MIT
- Text: This open source Python library provide several solvers for optimization problems related to